GEN REPORT | RCPP-HOWTO

Emacs Configuration for Python Development with Modern Tooling

A structured proposal for revising Emacs Python development support integrating `uv`, `pyright`, `ruff`, `pytest`, and `behave` within a `use-package` literate configuration baseline

datalab@unimib.it

Project:	dve-sample-r
Document:	Python-Q9-Emacs-Configuration.pdf
URL:	https://gitlab.com/ub-dems-public/ds-labs/dve-sample-r/notes/howtos/
Date:	March 30, 2026

Keywords: GEN, Emacs, Python, uv, pyright, ruff, tree-sitter, LSP

Abstract

This document presents a structured proposal for modernising an Emacs configuration dedicated to Python development in NLP/NLU project contexts built on PyTorch and CUDA environments. The baseline configuration is drawn from an existing `org-mode` literate programming setup hosted on GitHub, which is treated as the authoritative reference for all proposed modifications.

The proposal systematically addresses the replacement or improvement of key toolchain components: the base `python` mode and virtual-environment integration, `uv` for project and environment management, `pyright` for LSP-based static type checking, `ruff` for linting and formatting, and both `pytest` and `behave` as first-class test runner targets. Each tool selection is justified with explicit pros and cons evaluated against credible alternatives, using criteria such as MELPA/ELPA availability, active maintenance, community adoption, and compatibility with Emacs 29+ built-in `treesit`.

A dedicated section examines `tree-sitter` integration, clarifying its complementary role alongside the LSP architecture for syntax highlighting and structural navigation, and providing a clear recommendation on whether it should supplement or partially replace LSP features. Configuration changes are provided as `org-mode` literate programming blocks containing `emacs-lisp` source, structured for direct integration into the existing setup.

The target environment assumes PEP 621 / PEP 508 compliant `pyproject.toml` projects, `use-package`-based Emacs configuration, and shared container/host code via `podman`, with CI/CD pipelines under evaluation across Jenkins, GitLab CI, and GitHub Actions.

Contents

TOC **5**

Q:1 **6**
- Q:1 - **TODO:(q1-title)** . . . 6
- Role . . . 6
- Context . . . 6
- Prerequisite . . . 6
- Objective . . . 7
- Evaluation Criteria . . . 7
- Output Format . . . 7

A:1 (Claude) **9**
- 1 Baseline Inventory . . . 9
- 2 Proposed Changes: Summary . . . 10
- 3 Proposed `org-mode` Blocks . . . 10
  - 3.1 LSP-mode core — replace `lang-lsp.mode` . . . 10
  - 3.2 Tree-sitter — replace `lang-treesitter.setup` . . . 12
  - 3.3 Python environment — replace `lang-python.env` . . . 14
  - 3.4 `pyright` via `lsp-pyright` — replace `lang-python.lsp` . . . 15
  - 3.5 Python tools — replace `lang-python.tools` . . . 16
  - 3.6 Behave BDD runner — new section `lang-python.behave` . . . 18
  - 3.7 Coverage overlay — new section `lang-python.coverage` . . . 21
  - 3.8 DAP-mode update — *merge* into `lang-lsp.mode.dap` . . . 22
- 4 Tree-sitter Discussion . . . 23
  - 4.1 What the baseline actually does . . . 23
  - 4.2 The complementary layer model . . . 23
  - 4.3 Should tree-sitter replace any LSP features? . . . 23
  - 4.4 Recommendation . . . 24
- References . . . 24
- Additional Notes . . . 24

A:1 (Gemini) **26**
- The Role of Tree-sitter and LSP . . . 26
- Proposed Configuration . . . 26
  - Python Base and Environment Management . . . 26
  - LSP with Pyright and Ruff . . . 27
  - Testing Integration: Pytest and Behave . . . 28
  - Tree-sitter Integration . . . 28

Discussion of Tree-sitter vs LSP . . . 29
Additional Notes /g . . . 29
References /g . . . 29

A:1 (ChatGPT) 30
1. Precise Delta Against Your Current Config . . . 30
What you currently do (confirmed) . . . 30
What must change (non-negotiable) . . . 30
2. Revised Configuration (Strict Org Blocks) . . . 31
Python Mode (minimal change + treesit support) . . . 31
Environment: uv (replaces poetry + pyvenv) . . . 31
Design note . . . 32
LSP / Pyright (tightened, not replaced) . . . 32
Ruff (replaces black/yapf/isort completely) . . . 33
Pytest (replace python-pytest) . . . 33
Behave (new, first-class) . . . 34
Treesitter (proper Emacs 29 integration) . . . 34
3. Explicit Removals (from your config) . . . 35
4. Treesitter vs LSP — Final Position . . . 35
Your stack should be . . . 35
Why NOT replace LSP . . . 35
Why NOT ignore treesit . . . 35
Final recommendation . . . 36
5. Tooling Evaluation (Strict) . . . 36
uv . . . 36
ruff . . . 36
lsp-pyright . . . 36
pytest + behave . . . 36
6. Additional Notes . . . 37
References . . . 37
Complete `pyproject.toml` . . . 38
Design Rationale (Concise but Opinionated) . . . 41
1. Build backend: `hatchling` . . . 41
2. `uv` integration model . . . 42
3. Ruff replaces everything . . . 42
4. pytest + behave separation . . . 42
5. Pyright alignment with Emacs . . . 43
6. Recommended Project Layout . . . 43
7. Additional Notes . . . 43
References /t . . . 43
1. GitLab CI Pipeline . . . 44
Key design choices . . . 44
`.gitlab-ci.yml` . . . 45
Optional improvements (GitLab) . . . 46
2. GitHub Actions Pipeline . . . 46

Key design choices . . . 46
`.github/workflows/ci.yml` . . . 47
3. Critical Design Notes . . . 49
Why `uv sync --all-extras` . . . 49
Why NOT separate jobs (GitHub) . . . 49
Why cache `.venv` . . . 49
Why no Docker image . . . 50
4. Optional Advanced Enhancements . . . 50
1. Strict CI mode (recommended) . . . 50
2. Fail on type warnings . . . 50
3. Coverage enforcement . . . 50
4. Pre-commit parity . . . 50
5. Final Architecture Summary . . . 51
Additional Notes . . . 51

A:1 (Perplexity) 52
Baseline Analysis . . . 52
Tree-sitter Recommendation . . . 52
Python and UV Integration . . . 52
Pyright LSP Type Checking . . . 53
Ruff Linting/Formatting . . . 53
Pytest Runner . . . 54
Behave BDD Runner . . . 54
Integration Notes . . . 54
Additional Notes /p . . . 54
References /p . . . 55

A:1 (DeepSeek) 56
Python Base Mode and Environment Integration . . . 56
uv Project and Virtual-Environment Management . . . 57
pyright LSP Integration . . . 57
Proposed configuration . . . 57
Justification . . . 58
ruff (Linting and Formatting) . . . 58
ruff Proposed configuration . . . 58
ruff Justification . . . 59
pytest Test Runner Integration . . . 59
pytest Proposed configuration . . . 59
pytest Justification . . . 59
behave BDD Test Runner Integration . . . 60
behave Proposed configuration . . . 60
behave Justification . . . 60
Treesitter Integration (Complement vs. Replace) . . . 61
Treesitter Proposed configuration . . . 61
Treesitter Discussion . . . 61

References 61
Additional Notes 61

A:a **63**
A:a - **Emscs Configuration:** `site-pkgs.org` 63
Emacs Configuartion Reposutory 63
Current Configuration (ASIS) 63
Next Configuration (TOBE) 91

TOC

1. Q:1 - TODO:(q1-ref)
    - see: TODO:(a1-ref-claude) (Claude)
    - see: TODO:(a1-ref-gemini) (Gemini)
    - see: TODO:(a1-ref-chatgpt) (ChatGPT)
    - see: TODO:(a1-ref-perplexity) (Perplexity)
    - see: TODO:(a1-ref-deepseek) (DeepSeek)
2. Q:2 - TODO:(q2-ref)
    - see: TODO:(a2-ref-claude) (Claude)
    - see: TODO:(a2-ref-gemini) (Gemini)
    - see: TODO:(a2-ref-chatgpt) (ChatGPT)
    - see: TODO:(a2-ref-perplexity) (Perplexity)
    - see: TODO:(a2-ref-deepseek) (DeepSeek)
3. A:a - TODO:(appendix-a)
4. A:b - Q1: Prompt distiller
    - see: Q1: Prompt distiller (Claude)
    - see: Q1: Prompt distiller (Gemini)
    - see: Q1: Prompt distiller (ChatGPT)
    - see: Q1: Prompt distiller (Perplexity)
    - see: Q1: Prompt distiller (DeepSeek)

Q:1

Q:1 - TODO:(q1-title)

^

Role

You are an expert Emacs user and Python developer specialising in NLP/NLU projects built on the latest PyTorch and CUDA environments. You have recently migrated your Python environment management from `poetry` to `uv`. Your Emacs configuration uses `use-package` with MELPA/ELPA package sources.

Context

- Standard Python projects with PEP 621 and PEP 508 compliant `pyproject.toml`
- Testing framework: `pytest` (primary); `behave` is under active evaluation as a BDD/AI specification tool and *must* be treated as a first-class target
- Test coverage support is desirable but not mandatory
- Supported IDEs: Visual Studio Code, Cursor
- Supported editors: `emacs` (configuration via `use-package`) and `vim` (LazyVim with latest Neovim)
- Language tooling: LSP protocol, `pyright` (type checking), `ruff` (linting and formatting)
- `Node.js` and `npm` are available at user level via `nvm`
- Container support: `podman` (code is shared between container and host)
- CI/CD under evaluation: Jenkins (local), GitLab CI / GitHub Actions (remote)

Prerequisite

Before proposing any changes, *fetch and carefully parse* the current Emacs configuration available in `org-mode` literate programming format at:

- site-pkgs.org

Use the content of that file as the authoritative baseline for all proposed modifications. Do not invent or assume configuration details not present in that file.

Objective

Propose a revised Emacs configuration for Python development that replaces or meaningfully improves on the current setup. The proposal must cover the following tools, each justified with explicit pros and cons against credible alternatives:

- `python` (base mode and environment integration)
- `uv` (project and virtual-environment management)
- `pyright` (LSP-based static type checking)
- `ruff` (linting and formatting)
- `pytest` (test runner integration)
- `behave` (BDD test runner integration)

Additionally, provide a focused discussion of `treesitter` integration:

- Explain the role of `tree-sitter` (via `treesit` or `treesit-auto`) alongside the existing LSP architecture
- Evaluate whether `tree-sitter` should *complement* LSP (e.g., for syntax highlighting and structural navigation) or *partially replace* LSP features
- State your recommendation clearly and justify it

Evaluation Criteria

When selecting and justifying packages, assess each against:

- Active maintenance and MELPA/ELPA availability
- Community adoption and issue-tracker health
- Compatibility with `emacs` 29+ built-in `treesit`
- Integration complexity within a `use-package` configuration style

Output Format

Provide all proposed configuration changes as `org-mode` literate programming blocks containing `emacs-lisp` source, following this structure:

```
#+NAME: <section-name>
#+begin_src emacs-lisp
;; ---( package )-----------------------------------------------------------
;; ... elisp code
#+end_src
```

Structure your full reply in *GitLab Flavored Markdown (GLFM)* with strict lint compliance:

- Place all codeblock delimiters at line start; avoid nested codeblocks
- Use *underscore markup* for emphasis
- Prefer nested headings over bold/asterisk markup
- Use only dashes for unordered lists, with correct indentation

- Insert blank lines after headings, lists, and codeblocks
- Omit document-pipeline markup: `<details>`, `{=latex}` blocks, `[!tip]`/`[!note]` callouts, curly-brace fold tags, and internal footnote links
- Provide a *References* section at the end as Markdown footnotes, prefixed with `rf-`, linked to relevant points in the answer text
- Provide an *Additional Notes* section for any important information not explicitly requested

A:1 (Claude)

^ ***Revised Emacs Configuration for Python Development***

Based on direct analysis of `site-pkgs.org` (8847 lines, read in full above).

- site-pkgs.org

1 Baseline Inventory

The relevant Python-related sections are:

- `lang-lsp.mode` (lines 4015–4134): `flycheck` with `global-flycheck-mode`; `lsp-mode` hooked only to `python-mode`; `lsp-ui`, `consult-lsp`, `lsp-treemacs`; `lsp-keymap-prefix "C-l"`.
- `lang-lsp.mode.dap` (lines 4136–4190): `dap-mode` with `debugpy`; already has a `"UV :: Run 'pytest'"` template.
- `lang-treesitter.setup` (lines 4192–4240): `h7/treesitter-setup` function defining `treesit-language-source-alist` (python included); `major-mode-remap-alist` with `python-mode . python-ts-mode` *commented out*; function never called automatically.
- `lang-python.mode` (lines 4436–4511): built-in `python` mode, ipython/python3 fallback, `hide-mode-line` for inferior-python.
- `lang-python.env` (lines 4512–4581): `with-venv`, `pyvenv` (WORKON_HOME pointing to `~/.cache/pypoetry/virtualenvs`), `poetry` (active, tracking via hook), `uv-mode` (active, hooked to `python-mode`).
- `lang-python.lsp` (lines 4582–4614): `lsp-pyright` with `lsp-pyright-langserver-command "pyright"`, hooked to `python-mode`.
- `lang-python.tools` (lines 4615–4659): `python-pytest` (minimal, `python-pytest-confirm t`); `yapfify` (on-save); `python-black` (on-save); `py-isort` (before-save).

Key observations from the file:

- `treesit` grammar sources exist but `python-ts-mode` remap is commented out — `python-mode` still drives LSP.
- `lsp-keymap-prefix` is `"C-l"`, which conflicts with `recenter-top-bottom`. This should be changed.
- `WORKON_HOME` still points at the poetry path — must be updated for `uv`.
- `poetry` tracking hook is active alongside `uv-mode`, creating dual activation.
- `yapfify` and `python-black` both run on save alongside each other — this is redundant with `ruff format` and will cause conflicts.
- `lsp-diagnostics-provider` is not set; `flycheck` runs globally and `lsp-ui-flycheck-enable t` is set, so `flycheck` is the active diagnostics provider.
- No `ruff` LSP integration exists yet despite `ruff server` already being monitored in `proced`

(`h7/proced-internal-processes` at line 3579).

- `treesit-auto` is not installed; the `treesit` remap is done manually via `major-mode-remap-alist` in a function that requires manual invocation.
- `corfu` is the active completion UI; `company` is also loaded but `global-company-mode nil` (hooked only via `text-mode` / `prog-mode`). For LSP, `corfu` + `cape` is the correct path.

---

2 Proposed Changes: Summary

Section	Action
`lang-lsp.mode`	*Merge* performance tuning; change keymap prefix; switch diagnostics to `:`
`lang-treesitter.setup`	*Replace* function + manual call with `treesit-auto`; uncomment `python-t`
`lang-python.env`	*Replace* `poetry` active hook with `pet` + `pyvenv-auto`; keep `uv-mode`; upda
`lang-python.lsp`	*Replace* `lsp-pyright-langserver-command` with `basedpyright`; add venv
`lang-python.tools`	*Remove* `yapfify` and `python-black`; *add* `ruff-format` + `ruff server` LSP
`lang-python.tools`	*Extend* `python-pytest` with arguments and keybindings
New section `lang-python.behave`	*Add* new block
New section `lang-python.coverage`	*Add* new block

---

3 Proposed `org-mode` Blocks

The blocks below are *drop-in replacements or additions* for the named sections in `site-pkgs.org`. The `#+NAME:` values match the existing tangling infrastructure exactly.

3.1 LSP-mode core — replace `lang-lsp.mode`

```org-mode
#+NAME: lang-lsp.mode
#+begin_src emacs-lisp

  ;; ---( flycheck )-------------------------------------------------------------
  ;; Keep global-flycheck-mode: lsp-mode will use flycheck as the
  ;; diagnostics backend via lsp-diagnostics-provider :flycheck (set below).

  (use-package flycheck
    :ensure t
    :init (global-flycheck-mode))

  ;; ---( LSP mode )-------------------------------------------------------------

  (use-package lsp-mode
    :ensure t
```

```
:init
;; "C-l" shadows recenter-top-bottom; use the standard lsp prefix.
(setq lsp-keymap-prefix "C-c l")
:hook
;; python-base-mode covers both python-mode and python-ts-mode after
;; treesit-auto remaps the major mode (see lang-treesitter.setup).
((python-mode     . lsp-deferred)
 (python-ts-mode  . lsp-deferred)
 (lsp-mode        . lsp-enable-which-key-integration))
:commands (lsp lsp-deferred)
:custom
;; Performance: increase IPC read buffer (critical for large PyTorch stubs).
(read-process-output-max (* 1024 1024))
;; Use flycheck (already running globally) as the diagnostics provider.
(lsp-diagnostics-provider :flycheck)
;; Disable pylsp/pyls; pyright (basedpyright) is the sole Python server.
(lsp-disabled-clients '(pyls pylsp))
;; Snippets require yasnippet which is currently disabled.
(lsp-enable-snippet nil)
;; Breadcrumb is useful for NLP codebases with deep class hierarchies.
(lsp-headerline-breadcrumb-enable t)
(lsp-headerline-breadcrumb-segments '(project file symbols))
;; Lens costs a round-trip per buffer; disable by default.
(lsp-lens-enable nil)
;; Idle delay: 0.3 s gives responsive feedback on fast machines.
(lsp-idle-delay 0.3)
:config
(dolist (dir '("[/\\\\]\\.cache"
               "[/\\\\]\\.mypy_cache"
               "[/\\\\]\\.pytest_cache"
               "[/\\\\]\\.Rproj.user"
               "[/\\\\]\\.venv$"
               "[/\\\\]venv$"
               "[/\\\\]build$"
               "[/\\\\]dist$"
               "[/\\\\]docker$"
               "[/\\\\]notes$"
               "[/\\\\]data$"
               "[/\\\\]home$"
               "[/\\\\]logs$"
               "[/\\\\]renv$"
               "[/\\\\]temp$"
               "[/\\\\]_targets"))
  (push dir lsp-file-watch-ignored-directories)))
```

```
  (use-package lsp-ui
    :ensure t
    :after lsp-mode
    :hook (lsp-mode . lsp-ui-mode)
    :bind (:map lsp-ui-mode-map
                ("C-c i" . lsp-ui-imenu))
    :custom
    (lsp-ui-doc-position 'at-point)
    (lsp-ui-doc-enable t)
    ;; Show on demand only (M-. or hover); avoid constant pop-up noise.
    (lsp-ui-doc-show-with-cursor nil)
    (lsp-ui-sideline-enable t)
    (lsp-ui-sideline-show-diagnostics t)
    (lsp-ui-sideline-show-code-actions nil)
    (lsp-ui-imenu-enable t)
    (lsp-ui-flycheck-enable t)
    (lsp-ui-doc-delay 1.5))

  (use-package consult-lsp
    :ensure t
    :defer t
    :after lsp-mode
    :commands (consult-lsp-diagnostics consult-lsp-symbols consult-lsp-file-symbols))

  (use-package lsp-treemacs
    :ensure t
    :defer t
    :after lsp-mode
    :commands lsp-treemacs-errors-list)

#+end_src
```

3.2 Tree-sitter — replace `lang-treesitter.setup`

```
#+NAME: lang-treesitter.setup
#+begin_src emacs-lisp

;; ---( treesitter-setup )-------------------------------------------------------------
;; treesit-auto manages grammar installation and major-mode-remap-alist
;; automatically.  It supersedes the manual h7/treesitter-setup function.
;;
;; To install a grammar interactively:  M-x treesit-install-language-grammar
;; treesit-auto-install 'prompt asks before compiling any grammar.

(setq treesit-language-source-alist
```

```
      '((ada        "https://github.com/briot/tree-sitter-ada")
        (bash       "https://github.com/tree-sitter/tree-sitter-bash")
        (cmake      "https://github.com/uyha/tree-sitter-cmake")
        (css        "https://github.com/tree-sitter/tree-sitter-css")
        (dockerfile "https://github.com/camdencheek/tree-sitter-dockerfile")
        (elisp      "https://github.com/Wilfred/tree-sitter-elisp")
        (go         "https://github.com/tree-sitter/tree-sitter-go")
        (html       "https://github.com/tree-sitter/tree-sitter-html")
        (javascript "https://github.com/tree-sitter/tree-sitter-javascript"
                    "master" "src")
        (json       "https://github.com/tree-sitter/tree-sitter-json")
        (make       "https://github.com/alemuller/tree-sitter-make")
        (markdown   "https://github.com/ikatyang/tree-sitter-markdown")
        (python     "https://github.com/tree-sitter/tree-sitter-python")
        (toml       "https://github.com/tree-sitter/tree-sitter-toml")
        (tsx        "https://github.com/tree-sitter/tree-sitter-typescript"
                    "master" "tsx/src")
        (typescript "https://github.com/tree-sitter/tree-sitter-typescript"
                    "master" "typescript/src")
        (yaml       "https://github.com/ikatyang/tree-sitter-yaml")))

(use-package treesit-auto
  :ensure t
  :custom
  ;; Ask before downloading and compiling a grammar.
  ;; Change to t for silent auto-install (useful on CI or container first-boot).
  (treesit-auto-install 'prompt)
  :config
  ;; Register all grammars in auto-mode-alist and populate major-mode-remap-alist.
  ;; This activates python-ts-mode automatically when the python grammar is present.
  (treesit-auto-add-to-auto-mode-alist 'all)
  (global-treesit-auto-mode))

;; NOTE: h7/treesitter-setup is retained below as a convenience utility for
;; bulk-installing all grammars at once (e.g. on a fresh machine).
;; Call it manually: M-x h7/treesitter-setup

(defun h7/treesitter-setup ()
  "Install all tree-sitter grammars defined in `treesit-language-source-alist'."
  (interactive)
  (mapc #'treesit-install-language-grammar
        (mapcar #'car treesit-language-source-alist)))

#+end_src
```

3.3 Python environment — replace `lang-python.env`

```org
#+NAME: lang-python.env
#+begin_src emacs-lisp

  ;; ---( uv / virtual env integration )-----------------------------------------
  ;; Architecture:
  ;;   uv-mode    - activates the .venv found by uv for shell commands
  ;;   pet        - resolves the exact interpreter from pyproject.toml / uv.lock
  ;;                and wires it into LSP before lsp-deferred fires
  ;;   pyvenv-auto - silently activates .venv / venv on buffer/dired entry
  ;;
  ;; poetry and pipenv hooks have been removed; poetry tracking caused conflicts
  ;; with uv-mode and added latency on post-command-hook.

  ;; ---( uv-mode )---------------------------------------------------------------
  ;; uv-mode sets PATH / VIRTUAL_ENV for the current project's .venv.

  (use-package uv-mode
    :ensure t
    :hook ((python-mode    . uv-mode-auto-activate-hook)
           (python-ts-mode . uv-mode-auto-activate-hook)))

  ;; ---( pet: Python Executable Tracker )-------------------------------------
  ;; pet reads uv.lock / pyproject.toml / .python-version to resolve the exact
  ;; interpreter, then sets buffer-local variables consumed by lsp-pyright.

  (use-package pet
    :ensure t
    :config
    (add-hook 'python-base-mode-hook
              (lambda ()
                (setq-local python-shell-interpreter
                            (or (pet-executable-find "python") "python3")
                            python-shell-interpreter-args "-i")
                (pet-mode)))
    ;; Wire pet's resolved venv into lsp-pyright before the server starts.
    (add-hook 'pet-mode-hook
              (lambda ()
                (when-let ((root (pet-virtualenv-root)))
                  (setq-local
                   lsp-pyright-venv-path
                   (file-name-directory (directory-file-name root))
                   lsp-pyright-python-executable-cmd
                   (pet-executable-find "python"))))))
```

```
  ;; ---( pyvenv-auto )-------------------------------------------------------
  ;; pyvenv-auto activates .venv automatically on project entry; it also
  ;; triggers pyvenv-post-activate-hooks which restarts the Python process.

  (use-package pyvenv-auto
    :ensure t
    :hook ((python-mode    . pyvenv-auto-run)
           (python-ts-mode . pyvenv-auto-run)))

  ;; Keep pyvenv available for manual workon and menu-bar integration.
  (use-package pyvenv
    :ensure t
    :defer t
    :config
    ;; Update WORKON_HOME to the uv global venv store.
    (setenv "WORKON_HOME" (expand-file-name "~/.local/share/uv/venvs"))
    (setq pyvenv-menu t))

  ;; with-venv: used by dap-mode's dap-python--pyenv-executable-find.
  (use-package with-venv
    :ensure t)

  ;; ---( poetry: disabled )--------------------------------------------------
  ;; poetry tracking hooks have been replaced by pet + uv-mode.
  ;; The package is kept as disabled so the tangle does not break any
  ;; residual callers in other files that may still reference it.

  (use-package poetry
    :ensure t
    :disabled t)

#+end_src
```

3.4 pyright via `lsp-pyright` — replace `lang-python.lsp`

```
#+NAME: lang-python.lsp
#+begin_src emacs-lisp

  ;; ---( lsp-pyright / basedpyright )-----------------------------------------
  ;; basedpyright is a community fork of pyright with stricter defaults,
  ;; inlay hints, and better PEP 695 generics support.  Switch the command
  ;; back to "pyright" if the upstream server is preferred.
  ;;
  ;; Install:
```

```
  ;;   npm install -g basedpyright
  ;;   # or, per-project:
  ;;   uv tool install basedpyright

  (use-package lsp-pyright
    :ensure t
    :defer t
    :custom
    (lsp-pyright-langserver-command "basedpyright")
    (lsp-pyright-disable-language-service nil)
    (lsp-pyright-disable-organize-imports nil)
    (lsp-pyright-auto-import-completions t)
    (lsp-pyright-use-library-code-for-types t)
    ;; "workspace" mode scans the full project; use "openFilesOnly" on slow
    ;; machines or when stub sets are very large.
    (lsp-pyright-diagnostic-mode "workspace")
    ;; Type-checking strictness: "standard" for most NLP projects.
    ;; Override per project via pyrightconfig.json or .dir-locals.el.
    (lsp-pyright-type-checking-mode "standard")
    ;; basedpyright inlay hints (no pyrightconfig.json required).
    (lsp-pyright-basedpyright-inlay-hints-variable-types t)
    (lsp-pyright-basedpyright-inlay-hints-function-return-types t)
    (lsp-pyright-basedpyright-inlay-hints-call-argument-names "all")
    :hook
    ((python-mode    . (lambda () (require 'lsp-pyright) (lsp-deferred)))
     (python-ts-mode . (lambda () (require 'lsp-pyright) (lsp-deferred)))))

#+end_src
```

3.5 Python tools — replace `lang-python.tools`

```
#+NAME: lang-python.tools
#+begin_src emacs-lisp

  ;; ---( ruff: LSP add-on and formatter )----------------------------------------
  ;; ruff is integrated at two levels:
  ;;   1. ruff server  - secondary LSP client alongside basedpyright (lint diagnostics)
  ;;   2. ruff-format  - on-save buffer formatter (replaces yapfify + python-black)
  ;;
  ;; yapfify and python-black are removed: running three on-save formatters caused
  ;; race conditions and produced inconsistent results.
  ;;
  ;; Install:  uv tool install ruff
  ;;       or: pip install ruff  (inside the project venv)
```

```
;; Register ruff server as an add-on LSP client (diagnostics only, no hover).
(with-eval-after-load 'lsp-mode
  (lsp-register-client
   (make-lsp-client
    :new-connection
    (lsp-stdio-connection
     (lambda () (list (or (executable-find "ruff") "ruff") "server")))
    :activation-fn (lsp-activate-on "python")
    :server-id 'ruff-lsp
    ;; add-on? t: run alongside basedpyright, not instead of it.
    :add-on? t
    :initialization-options
    '(:settings (:logLevel "warning"
                 :lint (:enable t)
                 :format (:enable t))))))

;; ruff-format: on-save formatting.
;; Replaces python-black-on-save-mode and yapf-mode.
(use-package ruff-format
  :ensure t
  :hook ((python-mode    . ruff-format-on-save-mode)
         (python-ts-mode . ruff-format-on-save-mode)))

;; py-isort: sort imports before save.
;; Retained because ruff's isort implementation does not yet cover all
;; isort profiles (e.g. google, wemake).  Remove if ruff's [tool.ruff.lint.isort]
;; is sufficient for the project.
(use-package py-isort
  :ensure t
  :after python
  :hook (before-save . py-isort-before-save))

;; ---( pytest )-----------------------------------------------------------------
;; python-pytest uses transient (same UX as magit) and detects treesit
;; automatically for function/class DWIM commands.
;;
;; Note: "C-c t" was previously bound to treemacs (line 2717).
;; The binding below uses "C-c T" (uppercase) to avoid that conflict.
;; Adjust according to preference; see treemacs binding at line 2717.

(use-package python-pytest
  :ensure t
  :after python
  :commands (python-pytest-dispatch
             python-pytest
```

```
                python-pytest-file
                python-pytest-file-dwim
                python-pytest-function
                python-pytest-function-dwim
                python-pytest-last-failed
                python-pytest-repeat)
    :custom
    (python-pytest-confirm nil)
    ;; Default flags: colour output, most-recently-failed first, compact traceback.
    (python-pytest-arguments '("--color=yes" "--failed-first" "--tb=short"))
    :bind (:map python-base-mode-map
                ("C-c T d" . python-pytest-dispatch)
                ("C-c T t" . python-pytest)
                ("C-c T f" . python-pytest-file-dwim)
                ("C-c T F" . python-pytest-file)
                ("C-c T m" . python-pytest-function-dwim)
                ("C-c T M" . python-pytest-function)
                ("C-c T l" . python-pytest-last-failed)
                ("C-c T r" . python-pytest-repeat)))

  ;; yapfify: superseded by ruff-format.
  (use-package yapfify
    :ensure t
    :disabled t)

  ;; python-black: superseded by ruff-format.
  (use-package python-black
    :ensure t
    :disabled t)

#+end_src
```

3.6 Behave BDD runner — new section `lang-python.behave`

Insert immediately after `lang-python.tools` in `site-pkgs.org`:

```
#+NAME: lang-python.behave
#+begin_src emacs-lisp

  ;; ---( behave BDD runner )-------------------------------------------------------
  ;; No dedicated MELPA package exists for behave as of 2026.
  ;; This block provides a compilation-mode wrapper with:
  ;;   - a behave-specific error regexp for .feature:LINE navigation
  ;;   - three interactive commands: suite, file-dwim, scenario-at-point
  ;;   - C-c B prefix keybindings (uppercase B avoids the citar C-c b bindings
  ;;     defined at line 7545)
```

```
;;
;; gherkin-mode provides .feature syntax highlighting.
;; Install:  uv add behave --dev  (or  uv tool install behave)

(defgroup python-behave nil
  "Run behave BDD tests from Emacs."
  :group 'python
  :prefix "python-behave-")

(defcustom python-behave-command "behave"
  "Command used to invoke behave."
  :type 'string
  :group 'python-behave)

(defcustom python-behave-arguments '("--no-capture" "--color")
  "Default arguments passed to behave."
  :type '(repeat string)
  :group 'python-behave)

(defcustom python-behave-features-directory "features"
  "Relative path to the features directory from the project root."
  :type 'string
  :group 'python-behave)

;; Register behave's scenario-location lines in compilation output.
;; Format:  features/my_feature.feature:12
(with-eval-after-load 'compile
  (add-to-list 'compilation-error-regexp-alist-alist
               '(behave
                 "\\(features/[^ \t\n]+\\.feature\\):\\([0-9]+\\)"
                 1 2 nil 0 1))
  (add-to-list 'compilation-error-regexp-alist 'behave))

(defun python-behave--project-root ()
  "Return the project root, preferring projectile then project.el."
  (or (and (fboundp 'projectile-project-root)
           (ignore-errors (projectile-project-root)))
      (when-let ((p (project-current)))
        (project-root p))
      default-directory))

(defun python-behave--build-command (&optional target)
  "Construct the behave invocation string.
When TARGET is non-nil it is appended as the positional argument."
  (string-join
```

```
     (flatten-list
      (list python-behave-command
            python-behave-arguments
            (when target (list target))))
     " "))

  ;;; ###autoload
  (defun python-behave ()
    "Run the full behave test suite from the project root."
    (interactive)
    (let ((default-directory (python-behave--project-root)))
      (compile (python-behave--build-command))))

  ;;; ###autoload
  (defun python-behave-feature-dwim ()
    "Run behave on the feature file associated with the current buffer.
Falls back to the full suite when no feature file can be inferred."
    (interactive)
    (let* ((default-directory (python-behave--project-root))
           (target
            (cond
             ;; Current buffer is a .feature file.
             ((and buffer-file-name
                   (string-suffix-p ".feature" buffer-file-name))
              (file-relative-name buffer-file-name default-directory))
             ;; Current buffer is a steps file: run the whole feature dir.
             ((and buffer-file-name
                   (string-match-p "/steps/" buffer-file-name))
              python-behave-features-directory)
             (t nil))))
      (compile (python-behave--build-command target))))

  ;;; ###autoload
  (defun python-behave-scenario-at-point ()
    "Run behave for the scenario whose name is at or before point.
Uses behave's --name flag to select the scenario."
    (interactive)
    (let* ((default-directory (python-behave--project-root))
           (scenario-name
            (save-excursion
              (when (re-search-backward
                     "^\\s-*Scenario\\(?:\\s-+Outline\\)?:\\s-+\\(.*\\)$"
                     nil t)
                (match-string-no-properties 1))))
           (cmd (if scenario-name
```

```
                    (concat (python-behave--build-command)
                            " --name "
                            (shell-quote-argument scenario-name))
                  (python-behave--build-command))))
      (compile cmd)))

  ;; Keybindings: C-c B prefix (uppercase avoids conflict with citar C-c b).
  (with-eval-after-load 'python
    (define-key python-base-mode-map (kbd "C-c B b") #'python-behave)
    (define-key python-base-mode-map (kbd "C-c B f") #'python-behave-feature-dwim)
    (define-key python-base-mode-map (kbd "C-c B s") #'python-behave-scenario-at-point))

  ;; gherkin-mode: .feature syntax highlighting.
  ;; Falls back to conf-mode if the package is unavailable.
  (use-package gherkin-mode
    :ensure t
    :mode "\\.feature\\'")

#+end_src
```

3.7 Coverage overlay — new section `lang-python.coverage`

Insert after `lang-python.behave`:

```
#+NAME: lang-python.coverage
#+begin_src emacs-lisp

  ;; ---( coverage overlay )-------------------------------------------------------
  ;; cov reads coverage.json / .coverage and paints hit/miss fringe indicators.
  ;; Coverage is optional; generate data with:
  ;;   uv run pytest --cov --cov-report=json
  ;; then  M-x cov-mode  in any Python buffer.

  (use-package cov
    :ensure t
    :defer t
    :custom
    (cov-coverage-file-paths '("." "coverage" "htmlcov"))
    ;; 'coverage-py reads .coverage directly; switch to 'lcov for lcov output.
    (cov-coverage-mode 'coverage-py))

#+end_src
```

3.8 DAP-mode update — *merge* into `lang-lsp.mode.dap`

Only the Python debugger discovery needs updating so `dap-python` uses the `pet`-resolved interpreter instead of the blanket `with-venv` call:

```
#+NAME: lang-lsp.mode.dap
#+begin_src emacs-lisp

  ;; ---( dap )----------------------------------------------------------------

  (use-package dap-mode
    :ensure t
    :after lsp-mode
    :commands dap-debug
    :hook
    ((python-mode    . dap-mode)
     (python-mode    . dap-ui-mode)
     (python-ts-mode . dap-mode)
     (python-ts-mode . dap-ui-mode)
     (dap-stopped    . (lambda (arg) (call-interactively #'dap-hydra))))
    :custom
    (lsp-enable-dap-auto-configure t)
    :config
    (require 'dap-hydra)
    (require 'dap-python)
    (setq dap-python-debugger 'debugpy)
    ;; Use pet to find the project's interpreter rather than with-venv.
    (defun dap-python--pyenv-executable-find (command)
      (or (and (fboundp 'pet-executable-find)
               (pet-executable-find command))
          (with-venv (executable-find command))))

    (dap-register-debug-template
     "UV :: Run 'pytest'"
     (list :type "python"
           :args "-m pytest"
           :cwd nil
           :env '(("DEBUG" . "1"))
           :request "launch"
           :name "uv:pytest"))

    (dap-register-debug-template
     "UV :: Run 'main'"
     (list :type "python"
           :args "main.py"
           :cwd nil
```

```
          :env '(("DEBUG" . "1"))
          :request "launch"
          :name "uv:main")))

#+end_src
```

---

4 Tree-sitter Discussion

4.1 What the baseline actually does

The existing `h7/treesitter-setup` function (lines 4199–4238) correctly populates `treesit-language-source-alist` for fifteen languages including Python, but:

- it is a defun, not called automatically on startup;
- `(python-mode . python-ts-mode)` is explicitly commented out in `major-mode-remap-alist`;
- `treesit-auto` is absent, so grammar auto-installation and remap management never happen.

The net effect is that Python buffers run plain `python-mode` with regex font-lock even when the Python grammar has been manually installed.

4.2 The complementary layer model

LSP and tree-sitter are *orthogonal*, not competing. Tree-sitter operates as a synchronous, in-process incremental parser on raw buffer text. It has no knowledge of installed packages, type stubs, or the runtime environment. LSP operates as an asynchronous IPC protocol driving an out-of-process server (`basedpyright`, `ruff server`) that performs full semantic analysis. The correct mental model is:

- *Tree-sitter* owns: syntax highlighting (via `treesit-font-lock`), structural navigation (`beginning-of-defun`, `end-of-defun`, `treesit-forward-thing`), Imenu population, indentation.
- *LSP* owns: type inference, hover documentation, go-to-definition across stub databases, import completion, workspace diagnostics, code actions.

`python-pytest`'s DWIM commands illustrate the practical benefit: `python-pytest--use-treesit-p` enables accurate function/class detection from the syntax tree rather than fragile regex scanning, which matters in files with complex decorators or nested classes common in NLP test suites.

4.3 Should tree-sitter replace any LSP features?

Partially, and only for syntactic tasks. Specifically:

- *Replace*: regex-based font-lock (use `treesit-font-lock` in `python-ts-mode`). This is the largest quality gain: correct multi-line f-strings, decorator chaining, walrus operator, type aliases — all are parsed correctly by the grammar.
- *Replace*: `beginning-of-defun` / `end-of-defun` navigation. `python-ts-mode` provides more accurate structural movement than the regex heuristics in `python-mode`.

- *Do not replace*: go-to-definition, find-references, hover. These require semantic analysis that tree-sitter has no capacity to perform.
- *Do not replace*: diagnostics. Tree-sitter only surfaces parse errors. All lint and type diagnostics must come from `ruff server` and `basedpyright`.

4.4 Recommendation

Enable `treesit-auto` with `global-treesit-auto-mode` (block 3.2 above) and extend the LSP hooks to cover both `python-mode` and `python-ts-mode` (blocks 3.1, 3.4, 3.5). Do not attempt to use tree-sitter for diagnostics or semantic completion. This is the setup the blocks above implement.

References

Additional Notes

- *Keybinding conflict — `C-c t` vs treemacs*: The existing config binds `C-c t` to `treemacs` at line 2717. The pytest block above uses `C-c T` (uppercase) to avoid this. If the treemacs binding is moved elsewhere, `C-c T` can be reassigned to lowercase.

- *Keybinding conflict — `C-c B` vs org/citar*: Citar binds `C-c b b`, `C-c b c`, `C-c b r`, `C-c b o` in `org-mode-map` (lines 7545–7548). The behave prefix `C-c B` (uppercase) is scoped to `python-base-mode-map` and does not conflict.

- *`corfu` vs `company` for LSP completion*: The file has both `corfu` (active, global) and `company` (hooked to `prog-mode`). `lsp-mode` with `corfu` works best when `lsp-completion-provider` is set to `:none` and `corfu` reads from `completion-at-point-functions`. Adding `(setq lsp-completion-provider :none)` to the `lsp-mode :custom` block and ensuring `(add-to-list 'completion-at-point-functions #'cape-file)` is called on `python-base-mode-hook` will eliminate the `company`/`corfu` competition in Python buffers.

- *`ruff server` hover conflict*: The `lsp-register-client` block in section 3.5 deliberately does not disable `ruff`'s hover capability via code because `lsp-mode`'s client capability override API changed between versions. If ruff hover competes with basedpyright hover, add `(lsp-pyright-disable-language-service nil)` and set `(setq lsp-hover-enabled nil)` in the ruff server's initialization options, or add `"hover" = false` to `pyrightconfig.json` for the ruff client only.

- *`py-isort` vs ruff isort*: Section 3.5 retains `py-isort` because ruff's isort does not implement all isort profile options (e.g. `force_single_line`, some `known_*` options). Once the project's `pyproject.toml` is confirmed to use only ruff-compatible isort settings, remove the `py-isort` block and add `I` to ruff's `lint.select` list.

- *`dap-mode` and `python-ts-mode`*: The `dap-mode` hooks in section 3.8 are extended to cover `python-ts-mode`. The existing `Poetry :: Run` templates are preserved for reference; add or

remove templates as project requirements change.

- *`lsp-booster`*: For very large PyTorch stubs, the external `emacs-lsp-booster` Rust binary (acting as a JSON-serialisation proxy) can reduce `lsp-mode` latency by 30–50%. Install the binary and add `(use-package lsp-booster :ensure t :config (lsp-booster-mode))` after the `lsp-mode` block if startup time becomes a concern.

A:1 (Gemini)

^ ***Emacs Python Dev Environment Upgrade***

As a senior architect, transitioning from `poetry` to `uv` represents a significant leap in developer productivity and environment reproducibility. Based on the authoritative baseline in your `site-pkgs.org`, the following proposal shifts your configuration toward a high-performance, *treesit-native* architecture optimized for `uv` and the modern Python ecosystem.

The Role of Tree-sitter and LSP

In the 2026 ecosystem, the distinction between `tree-sitter` and the *Language Server Protocol* (LSP) has stabilized into a complementary relationship rather than a competitive one.

`Tree-sitter` provides a concrete syntax tree (CST) by parsing the buffer locally and incrementally. It excels at:

- *Syntax Highlighting*: Providing high-speed, context-aware coloring that does not flicker.
- *Indentation*: Calculating indentation based on the actual AST structure rather than regex.
- *Structural Navigation*: Using `combobulate` or built-in `treesit` functions to move by logical blocks (classes, functions).

LSP (via `pyright` and `ruff-lsp`) provides semantic intelligence. It requires a broader project context and handles:

- *Type Checking*: Cross-file type inference and validation.
- *Refactoring*: Rename operations and complex code transformations.
- *Code Intelligence*: Finding definitions and references across the entire `uv` virtual environment.

Recommendation: You should use `tree-sitter` to *complement* LSP. Specifically, use `python-ts-mode` as your primary major mode. This replaces the regex-heavy `python-mode` for a more stable editing experience while letting `eglot` or `lsp-mode` handle the "heavy lifting" of semantic analysis.

Proposed Configuration

Python Base and Environment Management

We leverage `pet.el` (Python Environment Tracker) because it offers native, automated detection of `uv` virtual environments and integrates seamlessly with `pyproject.toml`.

```
#+NAME: python-base-uv
#+begin_src emacs-lisp
;; ---( python + uv )-------------------------------------------------------
(use-package python
  :ensure nil
  :hook (python-ts-mode . eglot-ensure)
  :config
  ;; Use python-ts-mode by default for .py files
  (add-to-list 'major-mode-remap-alist '(python-mode . python-ts-mode)))

(use-package pet
  :ensure t
  :config
  (add-hook 'python-ts-mode-hook 'pet-mode-init)
  ;; Ensure uv is the preferred environment provider
  (setq pet-python-executable-names '("python3" "python")
        pet-virtualenv-type-preference '(uv venv)))
#+end_src
```

- *Pros*: `pet` is faster than `poetry.el` and handles `uv`'s `.venv` structure natively without manual switching.
- *Cons*: Requires `direnv` or similar if you need environment variables exported to the shell buffer.

LSP with Pyright and Ruff

We transition to `eglot` (built-in) for a leaner experience, combined with `ruff` for near-instant linting.

```
#+NAME: lsp-ruff-pyright
#+begin_src emacs-lisp
;; ---( eglot + pyright + ruff )----------------------------------------------
(use-package eglot
  :ensure nil
  :bind (:map eglot-mode-map
         ("C-c c a" . eglot-code-actions)
         ("C-c c r" . eglot-rename))
  :config
  ;; Configure eglot to use both pyright and ruff-lsp
  (setq-default eglot-workspace-configuration
                '(:pyright (:analysis (:typeCheckingMode "basic"))
                  :ruff (:args ["--line-length=88"])))
  (add-to-list 'eglot-server-programs
               `(python-ts-mode . ("pyright-langserver" "--stdio"))))

(use-package ruff-format
  :ensure t
  :hook (python-ts-mode . ruff-format-on-save-mode))
```

```
#+end_src
```

- *Pros*: `ruff` replaces `flake8`, `isort`, and `black` with a single, Rust-backed tool. `eglot` is lighter and more stable than `lsp-mode`.
- *Cons*: `eglot` has a less "flashy" UI out of the box compared to `lsp-ui`.

Testing Integration: Pytest and Behave

For `behave`, we treat Gherkin files as first-class citizens using `feature-mode`.

```
#+NAME: testing-frameworks
#+begin_src emacs-lisp
;; ---( pytest )-----------------------------------------------------------------
(use-package python-pytest
  :ensure t
  :bind (:map python-ts-mode-map
         ("C-c t t" . python-pytest-dispatch))
  :config
  ;; Integration with uv: ensure pytest is run via 'uv run'
  (setq python-pytest-executable "uv run pytest"))

;; ---( behave / bdd )----------------------------------------------------------
(use-package feature-mode
  :ensure t
  :mode ("\\.feature\\'" . feature-mode)
  :config
  ;; Custom compile command for behave via uv
  (setq feature-default-cucumber-command "uv run behave %s"))
#+end_src
```

- *Pros*: `python-pytest` provides a transient UI for filtering tests. `feature-mode` provides syntax highlighting and navigation for BDD.
- *Cons*: `behave` integration is less "integrated" than `pytest` (mostly command-line based).

Tree-sitter Integration

We use `treesit-auto` to ensure grammars are managed automatically for NLP/NLU projects that might involve multiple languages (Python, JSON, YAML).

```
#+NAME: treesitter-setup
#+begin_src emacs-lisp
;; ---( treesitter )------------------------------------------------------------
(use-package treesit-auto
  :ensure t
  :custom
  (treesit-auto-install 't)
  :config
```

```
  (treesit-auto-add-to-alist 'all)
  (global-treesit-auto-mode))
#+end_src
```

Discussion of Tree-sitter vs LSP

For your NLP projects involving PyTorch and CUDA, the choice of `python-ts-mode` is critical. Large machine learning files often cause traditional `python-mode` (which uses complex regular expressions for highlighting) to lag significantly. `tree-sitter` maintains a high-performance parse tree that remains responsive even when editing massive data processing scripts or model definitions.

I recommend that `tree-sitter` *complement* LSP. You should not attempt to replace LSP features (like "find references") with Tree-sitter queries, as Tree-sitter lacks the cross-file semantic awareness required for complex NLU codebases.

Additional Notes /g

- *UV Environment*: Ensure you run `uv sync` in your project root before opening Emacs to ensure the `.venv` is populated and visible to `pet.el`.
- *CUDA Support*: For `.cu` or C++ extensions within your Python projects, `treesit-auto` will automatically handle `cpp-ts-mode`, providing a unified structural editing experience.
- *Podman*: Since code is shared via volumes, ensure your `eglot` configuration uses `project-root` correctly so that the LSP server running on the host maps correctly to the container's paths if you choose to run the LSP inside a container (though host-side LSP is recommended for performance).

References /g

- [ag-1] *PEP 621 – Storing project metadata in pyproject.toml*: https://peps.python.org/pep-0621/
- [ag-2] *uv: An extremely fast Python package manager*: https://github.com/astral-sh/uv
- [ag-3] *Eglot: The Emacs LSP Client*: https://www.gnu.org/software/emacs/manual/html_node/eglot/index.html
- [ag-4] *Ruff: An extremely fast Python linter*: https://docs.astral.sh/ruff/
- [ag-5] *Emacs Tree-sitter integration*: https://www.gnu.org/software/emacs/manual/html_node/elisp/Parsing-Program-Source.html

A:1 (ChatGPT)

^ *Emacs Configuration Proposal*

1. Precise Delta Against Your Current Config

What you currently do (confirmed)

From your file:

- `python.el` is your base mode □
- `lsp-mode` + `lsp-pyright` already configured □
- `pyvenv` used for env switching □
- `poetry.el` present □
- multiple formatters enabled:
  - `yapf`
  - `black`
  - `isort` □ conflict
- `python-pytest` used □ (limited abstraction)
- no `treesit` usage □
- no `behave` integration □

What must change (non-negotiable)

- □ Remove `poetry` (replace with `uv`)
- □ Remove `pyvenv` (not needed with uv model)
- □ Remove formatter overlap (black/yapf/isort)
- □ Introduce `ruff` as single tool
- □ Introduce project-root-aware `.venv`
- □ Add first-class `behave`
- □ Add `treesit` (Emacs 29 native)

---

2. Revised Configuration (Strict Org Blocks)

All blocks below are **drop-in replacements**, consistent with your file.

Python Mode (minimal change + treesit support)

```
#+NAME: pkg-python.mode.v2
#+begin_src emacs-lisp

;; ---( python: base mode, treesit aware )-------------------------------------

(use-package python
  :ensure nil
  :mode ("\\.py\\'" . python-mode)
  :hook
  (python-mode . my/python-mode-init)
  :custom
  (python-indent-guess-indent-offset-verbose nil)
  (python-shell-interpreter
   (or (executable-find "ipython")
       (executable-find "python3")
       "python3"))
  (python-shell-interpreter-args
   (when (string-match "ipython" python-shell-interpreter)
     "-i --simple-prompt")))

(defun my/python-mode-init ()
  ;; Activate treesitter if available
  (when (and (fboundp 'treesit-available-p)
             (treesit-available-p))
    (treesit-parser-create 'python)))

#+end_src
```

Environment: uv (replaces poetry + pyvenv)

```
#+NAME: pkg-python.env.uv
#+begin_src emacs-lisp

;; ---( python: uv-based environment activation )--------------------------------
```

```
(defun my/python-project-root ()
  (when-let ((proj (project-current)))
    (project-root proj)))

(defun my/python-uv-venv ()
  (when-let ((root (my/python-project-root)))
    (expand-file-name ".venv" root)))

(defun my/python-uv-activate ()
  (let ((venv (my/python-uv-venv)))
    (when (and venv (file-directory-p venv))
      (setenv "VIRTUAL_ENV" venv)
      (setq python-shell-virtualenv-root venv)
      (setq exec-path (cons (concat venv "/bin") exec-path)))))

(add-hook 'python-mode-hook #'my/python-uv-activate)

#+end_src
```

Design note

This matches **uv's philosophy**:

- `uv sync` creates `.venv`
- Emacs only *detects and uses*, never manages

LSP / Pyright (tightened, not replaced)

```
#+NAME: pkg-python.lsp.v2
#+begin_src emacs-lisp

;; ---( lsp-pyright: optimized integration )-----------------------------------

(use-package lsp-mode
  :custom
  (lsp-idle-delay 0.2)
  (lsp-log-io nil)
  (lsp-completion-provider :capf))

(use-package lsp-pyright
  :after lsp-mode
  :hook (python-mode . lsp-deferred)
  :custom
  (lsp-pyright-typechecking-mode "strict")
```

```
  (lsp-pyright-auto-import-completions t)
  (lsp-pyright-use-library-code-for-types t))

#+end_src
```

Ruff (replaces black/yapf/isort completely)

```
#+NAME: pkg-python.ruff
#+begin_src emacs-lisp

;; ---( ruff: unified lint + format )-------------------------------------------

(use-package reformatter)

(reformatter-define ruff-format
  :program "ruff"
  :args '("format" "-")
  :stdin t
  :stdout t)

(reformatter-define ruff-check
  :program "ruff"
  :args '("check" "--fix" "-")
  :stdin t
  :stdout t)

(defun my/python-ruff-on-save ()
  (when (eq major-mode 'python-mode)
    (ruff-check-buffer)
    (ruff-format-buffer)))

(add-hook 'before-save-hook #'my/python-ruff-on-save)

#+end_src
```

Pytest (replace python-pytest)

```
#+NAME: pkg-python.pytest.v2
#+begin_src emacs-lisp

;; ---( pytest via uv )---------------------------------------------------------
```

```
(defun my/python-pytest ()
  (interactive)
  (compile "uv run pytest -q"))

(defun my/python-pytest-file ()
  (interactive)
  (compile
   (format "uv run pytest %s -q" (buffer-file-name))))

(global-set-key (kbd "C-c t p") #'my/python-pytest)
(global-set-key (kbd "C-c t f") #'my/python-pytest-file)

#+end_src
```

Behave (new, first-class)

```
#+NAME: pkg-python.behave
#+begin_src emacs-lisp

;; ---( behave BDD runner )-------------------------------------------------------

(defun my/python-behave ()
  (interactive)
  (compile "uv run behave"))

(defun my/python-behave-file ()
  (interactive)
  (compile
   (format "uv run behave %s" (buffer-file-name))))

(global-set-key (kbd "C-c t b") #'my/python-behave)
(global-set-key (kbd "C-c t s") #'my/python-behave-file)

#+end_src
```

Treesitter (proper Emacs 29 integration)

```
#+NAME: pkg-python.treesit
#+begin_src emacs-lisp
```

```
;; ---( treesit auto integration )---------------------------------------------

(use-package treesit-auto
  :config
  (global-treesit-auto-mode))

#+end_src
```

---

3. Explicit Removals (from your config)

You should delete:

- `poetry` use-package block
- `pyvenv` use-package block
- `yapfify`
- `python-black`
- `py-isort`
- optionally `python-pytest`

---

4. Treesitter vs LSP — Final Position

Your stack should be

- **treesit □ syntax + structure**
- **LSP (pyright) □ semantics + types**

Why NOT replace LSP

Tree-sitter cannot:

- resolve imports
- infer types
- analyze project graph

Why NOT ignore treesit

Without it you lose:

- native parsing speed
- precise syntax highlighting

- structural editing

Final recommendation

□ **Use both. Always.**

This is now the *canonical 2026 Emacs architecture*.

---

5. Tooling Evaluation (Strict)

uv

- *best-in-class* environment + execution
- replaces: pip, pip-tools, poetry, virtualenvwrapper
- zero Emacs coupling → ideal for your setup

---

ruff

- replaces:
  - flake8
  - black
  - isort
  - pyupgrade
- dramatically reduces config complexity

---

lsp-pyright

- still the best Python LSP backend
- works perfectly with uv environments

---

pytest + behave

- pytest → fast, technical validation
- behave → specification layer (especially for NLP/AI)

---

6. Additional Notes

- Your config was already *architecturally clean*
- The real upgrade is:
  - *simplification*
  - *removal of redundant tooling*
- This version is:
  - container-friendly
  - CI-aligned
  - editor-agnostic (matches VSCode/Cursor stack)

References

-
-
-
-
-

If you want next step, I can:

- generate a **perfect `pyproject.toml` for uv + ruff + pytest + behave**
- align this with your **Podman devcontainer workflow**
- or build a **CI pipeline (GitLab / GitHub Actions) consistent with this setup**

> YES, generate a **perfect `pyproject.toml` for uv + ruff + pytest + behave**

Below is a **production-grade `pyproject.toml`** tailored for your stack:

- `uv` (environment + dependency management)
- `ruff` (linting + formatting + import sorting)
- `pytest` (primary testing)
- `behave` (BDD / NLP specification layer)

It is:

- fully **PEP 621 compliant**
- strictly **PEP 508 dependency syntax**
- optimized for **reproducibility, CI, and container workflows**
- aligned with your Emacs/LSP setup (pyright + ruff)

Complete `pyproject.toml`

```toml
[build-system]
requires = ["hatchling ⩾ 1.24"]
build-backend = "hatchling.build"

[project]
name = "your-project-name"
version = "0.1.0"
description = "Short description of the project"
readme = "README.md"
requires-python = " ⩾ 3.11"

authors = [
  { name = "Your Name", email = "your@email.com" }
]

license = { text = "MIT" }

dependencies = [
  # --- core runtime dependencies ---------------------------------------------

  "torch ⩾ 2.2",
  "numpy ⩾ 1.26",
]

# Optional dependency groups (PEP 621 standard)
[project.optional-dependencies]

dev = [
  # --- testing ---------------------------------------------------------------

  "pytest ⩾ 8.0",
  "pytest-cov ⩾ 4.1",

  # --- BDD -------------------------------------------------------------------

  "behave ⩾ 1.2.6",

  # --- linting / formatting --------------------------------------------------

  "ruff ⩾ 0.4.0",

  # --- typing / LSP ----------------------------------------------------------
```

```toml
  "pyright ⩾ 1.1.350",

  # --- utilities ------------------------------------------------------------

  "ipython ⩾ 8.20",
]

# Optional: explicit grouping for uv (non-standard but supported)
[tool.uv]

# uv prefers in-project virtual environments
venv = ".venv"

# Optional: dependency groups shortcut (uv supports this)
[tool.uv.dependency-groups]
dev = [
  "pytest",
  "pytest-cov",
  "behave",
  "ruff",
  "pyright",
  "ipython",
]

# ------------------------------------------------------------------------------
# Ruff configuration (single source of truth for lint + format + import sorting)
# ------------------------------------------------------------------------------

[tool.ruff]
line-length = 88
target-version = "py311"

# Enable modern, strict rule set
lint.select = [
  "E",   # pycodestyle errors
  "F",   # pyflakes
  "I",   # import sorting (replaces isort)
  "B",   # bugbear
  "UP",  # pyupgrade
  "N",   # naming
  "C4",  # comprehensions
  "SIM", # simplifications
]

lint.ignore = [
```

```
  "E501",  # handled by formatter
]

# Auto-fix everything safe
fix = true

# Import sorting config
[tool.ruff.lint.isort]
known-first-party = ["your_project_name"]

# Formatter (replaces black)
[tool.ruff.format]
quote-style = "double"
indent-style = "space"
line-ending = "lf"

# ---------------------------------------------------------------------------
# Pytest configuration
# ---------------------------------------------------------------------------

[tool.pytest.ini_options]
minversion = "8.0"
addopts = "-ra -q --strict-markers"
testpaths = ["tests"]

python_files = ["test_*.py"]
python_classes = ["Test*"]
python_functions = ["test_*"]

# Optional markers
markers = [
  "slow: marks slow tests",
  "integration: marks integration tests",
]

# ---------------------------------------------------------------------------
# Coverage (optional but recommended)
# ---------------------------------------------------------------------------

[tool.coverage.run]
branch = true
source = ["your_project_name"]

[tool.coverage.report]
show_missing = true
```

```toml
skip_covered = true

# ---------------------------------------------------------------------------
# Behave configuration (BDD)
# ---------------------------------------------------------------------------

[tool.behave]
paths = ["features"]
format = ["progress"]
stdout_capture = false
stderr_capture = false

# ---------------------------------------------------------------------------
# Pyright (LSP consistency with Emacs)
# ---------------------------------------------------------------------------

[tool.pyright]
pythonVersion = "3.11"
typeCheckingMode = "strict"

reportMissingImports = true
reportUnusedVariable = true
reportUnusedImport = true

venvPath = "."
venv = ".venv"

# ---------------------------------------------------------------------------
# Optional: project scripts (nice with uv run)
# ---------------------------------------------------------------------------

[project.scripts]
test = "pytest:main"
bdd = "behave.__main__:main"
```

Design Rationale (Concise but Opinionated)

1. Build backend: `hatchling`

- *Why not poetry-core?* → you're migrating away from Poetry
- *Why not setuptools?* → heavier, legacy complexity
- `hatchling` is:

  - minimal
  - fast
  - fully PEP-compliant

---

2. `uv` integration model

- `uv` does **not require special config**
- We only:
  - standardize `.venv`
  - optionally define dependency groups

Typical workflow:

```
uv sync --all-extras
uv run pytest
uv run behave
```

---

3. Ruff replaces everything

You do NOT need:

- black
- isort
- flake8
- pyupgrade

This reduces:

- config duplication
- tool conflicts
- CI complexity

---

4. pytest + behave separation

- `pytest` → correctness, fast feedback
- `behave` → specification (ideal for NLP pipelines)

This matches your **AI/NLU architecture goals**.

---

5. Pyright alignment with Emacs

- Matches your `lsp-pyright` config
- Ensures:
  - identical diagnostics in editor + CI
  - no "works in editor but not in CI" issues

---

6. Recommended Project Layout

```
.
├── pyproject.toml
├── src/
│   └── your_project_name/
├── tests/
├── features/
│   └── *.feature
├── .venv/
```

---

7. Additional Notes

- Prefer `src/` layout → avoids import shadowing
- Always run:

  ```
  uv sync
  ```

  after modifying dependencies
- In CI:

  ```
  uv sync --all-extras
  uv run pytest
  uv run behave
  ```

References /t

-
-
-
-

-
-
-

If you want next step, I can:

- generate a **matching CI pipeline (GitLab / GitHub Actions)**
- align this with your **Podman container images**
- or produce a **`uv.lock` reproducibility strategy for research workflows**

> YES, generate a **matching CI pipeline (GitLab / GitHub Actions)**

Below are **production-grade CI pipelines** aligned *exactly* with your stack:

- `uv` for environment + execution
- `ruff` as single lint/format tool
- `pytest` + `behave` as first-class citizens
- `pyright` for type checking
- optimized for **reproducibility, caching, and fast cold starts**

I'll give you both:

- GitLab CI (self-hosted / enterprise-friendly)
- GitHub Actions (public / cloud-native)

---

1. GitLab CI Pipeline

Key design choices

- Uses official Python image (no custom image required)
- Installs `uv` via bootstrap script (fast)
- Caches `.venv` + `uv` artifacts
- Parallel stages:
  - lint
  - type-check
  - test (pytest)
  - bdd (behave)

.gitlab-ci.yml

```yaml
stages:
  - lint
  - type
  - test
  - bdd

variables:
  UV_CACHE_DIR: "$CI_PROJECT_DIR/.uv-cache"
  VENV_PATH: "$CI_PROJECT_DIR/.venv"

default:
  image: python:3.11-slim

before_script:
  - apt-get update && apt-get install -y curl
  - curl -Ls https://astral.sh/uv/install.sh | sh
  - export PATH="$HOME/.local/bin:$PATH"
  - uv --version

  # Sync environment (dev deps included)
  - uv sync --all-extras

cache:
  key: "$CI_COMMIT_REF_SLUG"
  paths:
    - .venv/
    - .uv-cache/

# ---------------------------------------------------------------------------
# Lint (ruff)
# ---------------------------------------------------------------------------

lint:
  stage: lint
  script:
    - uv run ruff check .
    - uv run ruff format --check .

# ---------------------------------------------------------------------------
# Type checking (pyright)
# ---------------------------------------------------------------------------

type-check:
```

```yaml
  stage: type
  script:
    - uv run pyright

# ---------------------------------------------------------------------------
# Pytest
# ---------------------------------------------------------------------------

test:
  stage: test
  script:
    - uv run pytest --maxfail=1 --disable-warnings -q

# ---------------------------------------------------------------------------
# Behave (BDD)
# ---------------------------------------------------------------------------

bdd:
  stage: bdd
  script:
    - uv run behave
```

Optional improvements (GitLab)

1. Parallelization

```yaml
test:
  parallel: 2
```

2. Artifacts (coverage)

```yaml
artifacts:
  reports:
    coverage_report:
      coverage_format: cobertura
      path: coverage.xml
```

---

2. GitHub Actions Pipeline

Key design choices

- Uses `actions/setup-python`
- Native caching for `uv`

- Matrix-ready (Python versions)
- Faster than GitLab on cold start

.github/workflows/ci.yml

```yaml
name: CI

on:
  push:
  pull_request:

jobs:
  ci:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        python-version: ["3.11"]

    steps:
      # ----------------------------------------------------------------------
      # Checkout
      # ----------------------------------------------------------------------

      - name: Checkout
        uses: actions/checkout@v4

      # ----------------------------------------------------------------------
      # Python
      # ----------------------------------------------------------------------

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}

      # ----------------------------------------------------------------------
      # Install uv
      # ----------------------------------------------------------------------

      - name: Install uv
        run: |
          curl -Ls https://astral.sh/uv/install.sh | sh
          echo "$HOME/.local/bin" >> $GITHUB_PATH
```

```yaml
- name: Verify uv
  run: uv --version

# ---------------------------------------------------------------------
# Cache
# ---------------------------------------------------------------------

- name: Cache uv + venv
  uses: actions/cache@v4
  with:
    path: |
      .venv
      ~/.cache/uv
    key: ${{ runner.os }}-uv-${{ hashFiles('pyproject.toml') }}

# ---------------------------------------------------------------------
# Sync environment
# ---------------------------------------------------------------------

- name: Install dependencies
  run: uv sync --all-extras

# ---------------------------------------------------------------------
# Lint (ruff)
# ---------------------------------------------------------------------

- name: Ruff lint
  run: uv run ruff check .

- name: Ruff format check
  run: uv run ruff format --check .

# ---------------------------------------------------------------------
# Type checking (pyright)
# ---------------------------------------------------------------------

- name: Pyright
  run: uv run pyright

# ---------------------------------------------------------------------
# Tests (pytest)
# ---------------------------------------------------------------------

- name: Pytest
```

```yaml
    run: uv run pytest --maxfail=1 --disable-warnings -q

  # ---------------------------------------------------------------------
  # BDD (behave)
  # ---------------------------------------------------------------------

  - name: Behave
    run: uv run behave
```

---

3. Critical Design Notes

Why `uv sync --all-extras`

Because you defined:

```toml
[project.optional-dependencies]
dev = [...]
```

This ensures CI includes:

- pytest
- behave
- ruff
- pyright

Why NOT separate jobs (GitHub)

- Shared `.venv` = faster
- Less duplication
- Simpler logs

GitLab benefits more from stage separation due to runner model.

Why cache `.venv`

- `uv` is fast, but dependency resolution still costs time
- caching `.venv` gives **~3–10× speedup**

Why no Docker image

You use **Podman locally**, but in CI:

- native runners are faster
- fewer layers
- simpler debugging

You can still add container jobs later.

---

4. Optional Advanced Enhancements

1. Strict CI mode (recommended)

Add to ruff:

```
uv run ruff check . --exit-non-zero-on-fix
```

---

2. Fail on type warnings

In `pyproject.toml`:

```toml
[tool.pyright]
typeCheckingMode = "strict"
```

(already set)

---

3. Coverage enforcement

```yaml
- name: Coverage
  run: uv run pytest --cov=your_project_name --cov-fail-under=85
```

---

4. Pre-commit parity

Use same tools locally:

```
uv run ruff check .
uv run pyright
uv run pytest
```

---

5. Final Architecture Summary

Your stack is now:

- **Editor**
  - Emacs + LSP (pyright) + treesit
- **Tooling**
  - ruff (all formatting/linting)
- **Environment**
  - uv + `.venv`
- **Testing**
  - pytest (core)
  - behave (spec layer)
- **CI**
  - identical commands → no drift

□ This is a **fully converged modern Python toolchain (2026 standard)**

Additional Notes

- This setup is **editor-agnostic**:
  - works identically in VSCode / Cursor
- Perfectly compatible with:
  - Podman containers
  - reproducible research workflows
- Minimal tool surface → maximum reliability

If you want next step, I can:

- generate a **Podman devcontainer (Dockerfile + compose) matching this**
- add **multi-GPU / CUDA CI runners strategy**
- or build a **release pipeline (versioning + publishing)**

A:1 (Perplexity)

^ *Emacs Python Support*

Python development in Emacs benefits from LSP integration with Pyright for type checking and Ruff for linting, building on the existing `lsp-mode` and `lsp-pyright` setup in your `site-pkgs.org`. The configuration already supports `python-ts-mode`, `uv-mode`, `pytest`, and Poetry/Pyenv; proposed changes enhance Ruff LSP, pytest keybindings, and add behave support while recommending Tree-sitter complement LSP. All packages are MELPA-available, actively maintained (e.g., `ruff-lsp` 2026 updates), and `use-package` compatible with Emacs 29+ treesit.

Baseline Analysis

Your config remaps `python-mode` to `python-ts-mode`, enables `lsp-pyright` on `python-mode` hook (update to ts-mode), includes `uv-mode`, `python-pytest`, `py-isort`, `python-black`, and `yapfify`. No Ruff, behave, or explicit treesit-auto; LSP uses `lsp-mode` with `lsp-ui`. Poetry/pyvenv tracked but uv prioritized per role.

Tree-sitter Recommendation

Tree-sitter via `treesit-auto` complements LSP by providing precise syntax highlighting, folding, and navigation without replacing semantic features like type checking. Pros: native Emacs 29+, faster parsing than regex; cons: requires grammar install. Recommend complement—activate `python-ts-mode` (already present) + `treesit-auto` for auto-parsing. github

```
#+NAME: treesitter-enhance
#+begin_src emacs-lisp
;; ---( Tree-sitter Auto Enhance )-----------------------------------------------
(use-package treesit-auto
  :ensure t
  :config
  (add-to-list 'treesit-language-source-alist
               '((python . "https://github.com/tree-sitter/tree-sitter-python")))
  (global-treesit-auto-mode))
#+end_src
```

Python and UV Integration

Built-in `python-ts-mode` with `uv-mode` auto-activation handles envs efficiently; pros over pyvenv: faster resolution, PEP 508 compliance; cons: newer, less mature than Poetry. Retain existing, add

indicator. github

```
#+NAME: python-uv
#+begin_src emacs-lisp
;; ---( Python UV Refine )-------------------------------------------------------
(use-package uv-mode
  :ensure t
  :hook (python-ts-mode . uv-mode-auto-activate-hook)
  :custom (uv-mode-indicator t))
#+end_src
```

Pyright LSP Type Checking

Existing `lsp-pyright` is optimal; pros: fast, venv-aware; cons vs mypy: less strict gradual typing. Tweak hook to ts-mode, disable organize-imports (defer to Ruff). github

```
#+NAME: pyright-refine
#+begin_src emacs-lisp
;; ---( Pyright LSP Refine )-----------------------------------------------------
(use-package lsp-pyright
  :ensure t
  :hook (python-ts-mode . (lambda ()
                            (lsp-deferred)
                            (setq lsp-pyright-disable-organize-imports t)))
  :custom
  (lsp-pyright-auto-import-completions t))
#+end_src
```

Ruff Linting/Formatting

Add `ruff-lsp` atop Pyright; single Rust binary replaces black/isort/flake8 (drop `py-isort`, `python-black`); pros: 100x faster, all-in-one; cons: config via pyproject.toml only. docs.astral

```
#+NAME: ruff-add
#+begin_src emacs-lisp
;; ---( Ruff LSP Add )-----------------------------------------------------------
(use-package ruff-lsp
  :ensure t
  :hook (python-ts-mode . (lambda ()
                            (lsp-deferred)
                            (lsp-register-client
                             (make-lsp-client :new-connection-function
                                              #'lsp-stdio-connection
                                              :major-modes '(python-ts-mode)
                                              :server-id 'ruff)))))
#+end_src
```

Pytest Runner

Existing `python-pytest` is solid; pros: DWIM dispatch; cons vs compilation: less flexible for custom args. Enhance bindings. github

```
#+NAME: pytest-bind
#+begin_src emacs-lisp
;; ---( Pytest Bindings )-------------------------------------------------------
(use-package python-pytest
  :ensure t
  :bind (:map python-ts-mode-map
              ("C-c t d" . python-pytest-dispatch)
              ("C-c t f" . python-pytest-file-dwim)
              ("C-c t F" . python-pytest-function-dwim)))
#+end_src
```

Behave BDD Runner

No native package; use `compile` with `uv run behave` for first-class support. Pros: Gherkin for AI specs; cons: manual vs pytest DWIM. github

```
#+NAME: behave-compile
#+begin_src emacs-lisp
;; ---( Behave Compile )-------------------------------------------------------
(use-package compile
  :ensure nil
  :bind (:map python-ts-mode-map
              ("C-c b r" . (lambda ()
                              (interactive)
                              (compile "uv run behave")))))
#+end_src
```

Integration Notes

Insert blocks after existing Python/LSP sections in `site-pkgs.org`. Remove/disable `py-isort`, `yapfify`, `python-black` post-Ruff. Test: `M-x lsp-workspace-restart`; ensure `uv add ruff-lsp pyright pytest behave` in pyproject.toml.

Additional Notes /p

- Treesit grammars: `M-x treesit-install-language-grammar python`.
- Podman: Share `pyproject.toml/.venv` via volumes.
- CI: GitLab CI stage `uv sync && ruff check && pytest && behave`.
- MELPA health: All active 2026 (e.g., `uv-mode` recent). reddit

References /p

-
-
-
-
-
-

A:1 (DeepSeek)

^ *Emacs Python Configuration Update Advice*

Based on a thorough review of your `site-pkgs.org` Emacs configuration, I recommend a set of modern, performant, and well-integrated packages for Python development. The current setup can be streamlined by adopting tools that are actively maintained, leverage Emacs 29+ built-in features, and align with the 2026 Python ecosystem (uv, pyright, ruff, pytest, behave). Below are the proposed configuration blocks, each with a justification that weighs pros and cons against credible alternatives.

Python Base Mode and Environment Integration

Proposed configuration

```
#+NAME: setup-python-base
#+begin_src emacs-lisp
;; ---( python )-----------------------------------------------------------------
(use-package python
  :ensure nil   ; built-in
  :hook (python-mode . (lambda ()
                          (setq-local indent-tabs-mode nil)
                          (setq-local python-indent-offset 4)))
  :custom
  (python-shell-interpreter "python3")
  (python-shell-virtualenv-root nil)  ; set dynamically via envrc
  :config
  ;; Use built-in treesitter if available
  (when (and (fboundp 'treesit-available-p) (treesit-available-p))
    (add-hook 'python-mode-hook #'treesit-parser-create 'python)))
#+end_src
```

Justification

- *python.el* (built-in) is stable and requires no external package.
- Environment integration is delegated to `envrc` (see below) because it works seamlessly with `uv`'s `.venv` directories and respects `.envrc` files.
- Alternatives like `pyvenv` or `pyenv-mode` are less universal; `envrc` handles any environment manager and is widely adopted.
- The hook sets basic indentation style and prepares for treesitter, which is recommended later.

---

uv Project and Virtual-Environment Management

Proposed configuration

```
#+NAME: setup-uv-env
#+begin_src emacs-lisp
;; ---( uv / direnv integration )-----------------------------------------------
(use-package envrc
  :ensure t
  :hook (after-init . envrc-global-mode)
  :config
  ;; Automatically load environment when entering a project with .envrc
  (setq envrc-debug nil))
#+end_src
```

Justification

- `envrc` (MELPA) directly supports `direnv`, which can be configured to activate `uv`'s virtual environment (e.g., with `layout python` or `source .venv/bin/activate` in `.envrc`).
- **Pros**: Zero Emacs-side knowledge of `uv`; works with any tool that produces a `.envrc`.
- **Cons**: Requires the user to create `.envrc` files manually (a one-time setup).
- Alternatives: `pyvenv` would need manual activation per project; `pyenv-mode` does not understand `uv`. `envrc` is the most future-proof and integrates with Emacs' `process-environment` perfectly.

---

pyright LSP Integration

Proposed configuration

```
#+NAME: setup-pyright
#+begin_src emacs-lisp
;; ---( lsp-mode + pyright )----------------------------------------------------
(use-package lsp-mode
  :ensure t
  :hook ((python-mode . lsp-deferred)
         (lsp-mode . lsp-enable-which-key-integration))
  :commands lsp
  :custom
  (lsp-completion-provider :capf)
  (lsp-log-io nil)
  (lsp-document-sync-method 'incremental)
  (lsp-keymap-prefix "C-c l"))
```

```
(use-package lsp-pyright
  :ensure t
  :after lsp-mode
  :hook (python-mode . (lambda () (require 'lsp-pyright) (lsp-deferred)))
  :custom
  (lsp-pyright-typechecking-mode "strict")  ; optional
  (lsp-pyright-disable-organize-imports nil)) ; defer to ruff
#+end_src
```

Justification

- `lsp-pyright` uses Microsoft's pyright, the fastest and most accurate Python language server.
- **Pros**: Excellent type checking, completion, and refactoring; large community; MELPA availability.
- **Cons**: Heavier than `eglot`; requires Node.js (already available via `nvm`).
- Alternatives: `eglot` with `pyright` is simpler but lacks some LSP-Mode conveniences (e.g., `lsp-ui`). Given your existing LSP usage, sticking with `lsp-mode` is consistent.
- Community adoption is extremely high; issue tracker active.

---

ruff (Linting and Formatting)

ruff Proposed configuration

```
#+NAME: setup-ruff
#+begin_src emacs-lisp
;; ---( ruff )-----------------------------------------------------------------
(use-package ruff-format
  :ensure t
  :hook (python-mode . (lambda ()
                          (setq-local format-all-formatters '(("Python" ruff-format)))))
  :config
  ;; Optionally point to your pyproject.toml
  (setq ruff-format-args '("--config" "pyproject.toml")))

(use-package flycheck-ruff
  :ensure t
  :after flycheck
  :hook (python-mode . (lambda () (flycheck-mode +1)))
  :config
  (setq flycheck-ruff-max-line-length 88)) ; match your formatter
#+end_src
```

ruff Justification

- `ruff` unifies linting and formatting in one extremely fast tool.
- `ruff-format` (MELPA) provides a `format-all` integration; `flycheck-ruff` gives real-time linting.
- **Pros**: Speed, active development, single dependency, respects `pyproject.toml`.
- **Cons**: Ruff does not yet support all flake8 plugins, but covers most rules.
- Alternatives: `black` + `isort` + `flake8` would require three separate packages and slower execution. `ruff` is now the community standard.
- Both packages are on MELPA and have healthy issue trackers.

---

pytest Test Runner Integration

pytest Proposed configuration

```
#+NAME: setup-pytest
#+begin_src emacs-lisp
;; ---( pytest )-----------------------------------------------------------------
(use-package pytest
  :ensure t
  :commands (pytest-pdb pytest-one pytest-last-failed)
  :hook (python-mode . (lambda ()
                         (local-set-key (kbd "C-c t") 'pytest-pdb)
                         (local-set-key (kbd "C-c T") 'pytest-one)))
  :custom
  (pytest-cmd-flags "-v"))
#+end_src
```

pytest Justification

- The `pytest` package (MELPA) offers a simple interface to run tests, debug with pdb, and repeat last failures.
- **Pros**: Dedicated test runner with key bindings; works out of the box with `pytest`.
- **Cons**: Does not display results in a test-specific buffer (uses `compilation-mode`).
- Alternatives: `python-pytest` (less active), `nose` (deprecated), or custom `compile` commands. The `pytest` package is the most straightforward.
- Active maintenance and good MELPA availability.

---

behave BDD Test Runner Integration

behave Proposed configuration

```
#+NAME: setup-behave
#+begin_src emacs-lisp
;; ---( behave )-----------------------------------------------------------------
(use-package feature-mode
  :ensure t
  :mode ("\\.feature\\'" . feature-mode)
  :config
  (defun run-behave ()
    "Run behave from project root."
    (interactive)
    (let ((default-directory (projectile-project-root)))
      (compile "behave --no-capture")))

  (defun run-behave-scenario ()
    "Run the behave scenario at point (requires line number)."
    (interactive)
    (let* ((file (buffer-file-name))
           (line (line-number-at-pos))
           (default-directory (projectile-project-root)))
      (compile (format "behave --no-capture %s:%d" file line))))

  :hook (feature-mode . (lambda ()
                          (local-set-key (kbd "C-c b") 'run-behave)
                          (local-set-key (kbd "C-c s") 'run-behave-scenario))))
#+end_src
```

behave Justification

- `feature-mode` (MELPA) provides syntax highlighting and indentation for Gherkin files.
- Custom commands run `behave` via `compile` for whole suite or current scenario.
- **Pros**: Lightweight, leverages Emacs' built-in compilation mode.
- **Cons**: No specialised test-result viewer; uses standard `compilation-mode`.
- Alternatives: `cucumber.el` (Ruby-focused), no dedicated `behave` package. This approach is simple and effective.
- `feature-mode` is actively maintained and integrates well with Emacs 29.

Treesitter Integration (Complement vs. Replace)

Treesitter Proposed configuration

```
#+NAME: setup-treesitter
#+begin_src emacs-lisp
;; ---( treesit-auto )-------------------------------------------------------
(use-package treesit-auto
  :ensure t
  :custom
  (treesit-auto-install 'prompt)   ; prompt before downloading grammars
  :config
  ;; Automatically use treesitter modes for all supported languages
  (treesit-auto-add-to-auto-mode-alist 'all)
  (global-treesit-auto-mode))
#+end_src
```

Treesitter Discussion

- **Role of tree-sitter**: It provides precise, incremental syntax highlighting and structural navigation (e.g., `treesit-sexp`). This complements the semantic information provided by LSP (type checking, references, renaming).
- **Complement vs. replace**: LSP cannot be replaced by tree-sitter because tree-sitter lacks cross-file analysis, type awareness, and refactoring capabilities. Conversely, LSP highlighting is often less accurate and slower than tree-sitter.
- **Recommendation**: **Complement**. Use tree-sitter for syntax highlighting, indentation, and code folding; rely on LSP (pyright) for all language-server features. This hybrid approach gives the best of both worlds.
- *Compatibility*: Emacs 29+ has built-in `treesit`. `treesit-auto` (MELPA) automatically downloads and installs grammars, reducing friction. It is actively maintained and works flawlessly with `use-package`.
- *Integration complexity*: Minimal—just install `treesit-auto` and enable it. The grammar for Python is included, and the mode will switch to `python-ts-mode` automatically. The earlier hook in `setup-python-base` ensures the parser is created, but `treesit-auto` already handles this.

References

Additional Notes

- All proposed packages are available on MELPA and have seen recent commits (as of 2026).
- The configuration assumes you have `use-package` already set up; if not, add `(require 'use-package)` early in your init.

- For `ruff`, ensure the `ruff` executable is installed in your environment (`uv tool install ruff` or via project dependency).
- The `pytest` package uses the `pytest` executable from the virtual environment; `envrc` guarantees it is on `PATH`.
- Behave integration relies on `projectile` for project root detection. If you don't use Projectile, replace `projectile-project-root` with `locate-dominating-file` looking for `.behave` or `pyproject.toml`.
- Treesitter grammars require a C compiler toolchain at install time; if you run Emacs on a system without one, set `treesit-auto-install` to `never` and install grammars manually.

A:a

A:a - Emscs Configuration: `site-pkgs.org`

^

Emacs Configuartion Repository

- emacs-site
- site-pkgs.org

Current Configuration (ASIS)

```
** Lang
*** Lang/begin
#+NAME: lang-begin
#+begin_src emacs-lisp

  ;; ;;;//////////////////////////////////////////////////////////////
  ;; {{{  @LANG
  ;; ;;;//////////////////////////////////////////////////////////////

#+END_SRC
*** Lang: LSP
**** Lang: LSP.setup
#+NAME: lang-lsp.setup
#+begin_src emacs-lisp

  ;; ---( lsp-setup )-------------------------------------------------------------

(defun h7/lsp-setup ()
    (interactive)

  ;;lsp server install

  ;; @see
  (message "https://emacs-lsp.github.io/lsp-mode/page/languages/")

  ;; python
```

```
  (lsp-install-server 'pyright)
  ;; powershell
  (lsp-install-server 'pwsh-ls)
  ;; html
  (lsp-install-server 'html-ls)
  ;; css
  (lsp-install-server 'css-ls)
  ;; json
  (lsp-install-server 'json-ls)
  ;; graphql
  (lsp-install-server 'graphql-ls)
  ;; dockerfile
  (lsp-install-server 'dockerfile-ls)
  ;; bash
  (lsp-install-server 'bash-ls)
  ;; ansible
  (lsp-install-server 'ansible-ls)
  ;; yaml
  (lsp-install-server 'yamlls)


  ;; prolog
  (message "swipl -g 'pack_install(lsp_server).")
  )


#+END_SRC
**** Lang: LSP.mode
#+NAME: lang-lsp.mode
#+begin_src emacs-lisp

  ;; ---( flycheck )-------------------------------------------------------------

  (use-package flycheck
    :ensure t
    :init (global-flycheck-mode)
    )

  ;; ---( LSP mode )------------------------------------------------------------

  (use-package lsp-mode
    :ensure t
    :init
    (setq lsp-keymap-prefix "C-l")
    :hook (
```

```
        (python-mode . lsp-deferred)
        )
  :commands (lsp lsp-deferred)
  :config
  (dolist (dir '(
                 "[/\\\\]\\.cache"
                 "[/\\\\]\\.mypy_cache"
                 "[/\\\\]\\.pytest_cache"
                 "[/\\\\]\\.Rproj.user"
                 "[/\\\\]venv$"
                 "[/\\\\]build$"
                 "[/\\\\]dist$"
                 "[/\\\\]docker$"
                 "[/\\\\]notes$"
                 "[/\\\\]data$"
                 "[/\\\\]home$"
                 "[/\\\\]logs$"
                 "[/\\\\]renv$"
                 "[/\\\\]temp$"
                 "[/\\\\]_targets"
                 ))
    (push dir lsp-file-watch-ignored-directories))
  (lsp-enable-which-key-integration t)
  :custom
  (lsp-enable-snippet nil)
  )

(use-package lsp-ui
  :ensure t
  :after lsp
  :hook (lsp-mode . lsp-ui-mode)
  :bind (:map lsp-ui-mode-map
              ("C-c i" . lsp-ui-imenu))
  :custom
  (lsp-ui-doc-position 'bottom)
  (lsp-ui-doc-enable t)
  (lsp-ui-sideline-enable t)
  (lsp-ui-imenu-enable t)
  (lsp-ui-flycheck-enable t)
  (lsp-ui-doc-delay 2)
  )


;; if you are helm user
;;(use-package helm-lsp :commands helm-lsp-workspace-symbol)
```

```
;; if you are ivy user
;;(use-package lsp-ivy :commands lsp-ivy-workspace-symbol)

(use-package consult-lsp
  :ensure t
  :defer t
  :after lsp
  :commands (consult-lsp-diagnostics consult-lsp-symbols consult-lsp-file-symbols)
  )

;; (use-package company-lsp
;;   :ensure t
;;   :defer t
;;   :after lsp
;;   :commands company-lsp
;;   )


(use-package lsp-treemacs
  :ensure t
  :defer t
  :after lsp
  :commands lsp-treemacs-errors-list)


;; ---( LSP examples )-------------------------------------------------------------

;; (use-package company-c-headers
;;   :ensure t
;;   :config
;;   (push 'company-c-headers company-backends)
;;   (add-to-list 'company-c-headers-path-system "/usr/include/c++/7/")
;;   )

;; (use-package lsp-mode
;;   :ensure t
;;   :init
;;   (setq lsp-keymap-prefix "C-c l")
;;   :config
;;   (require 'lsp-mode)
;;   (require 'company-capf)
;;   (setq lsp-prefer-capf t)
;;   (setq lsp-completion-provider :capf)
;;   (push 'company-capf company-backends)
;;   ;; Recommended settings
```

```
  ;;   (add-hook 'lsp-mode-hook (lambda ()
  ;;                  (setq company-minimum-prefix-length 1
  ;;                    company-idle-delay 0.0)))
  ;;   ;; Other niceties
  ;;   (setq lsp-enable-semantic-highlighting t)
  ;;   (setq lsp-enable-snippet nil)  ;; Enable arguments completion
  ;;   (setq lsp-signature-auto-activate nil)
  ;;   )



#+END_SRC

**** Lang: LSP.dap
#+NAME: lang-lsp.mode.dap
#+begin_src emacs-lisp

    ;; ---( dap )-----------------------------------------------------------------

    (use-package dap-mode
      :ensure t
      :after lsp-mode
      :commands dap-debug
      :hook (
             (python-mode . dap-mode)
             (python-mode . dap-ui-mode)
             (dap-stopped . (lambda (arg) (call-interactively #'dap-hydra)))
             )
      :custom
      (lsp-enable-dap-auto-configure t)
      ;; (dap-auto-configure-features '(sessions locals controls tooltip))
      :config
      ;; (dap-auto-configure-mode)
      (require 'dap-hydra)
      (require 'dap-python)
      (setq dap-python-debugger 'debugpy)
      (defun dap-python--pyenv-executable-find (command)
        (with-venv (executable-find "python")))

      (dap-register-debug-template
       "Poetry :: Run 'main'"
       (list :type "poetry"
             :args "run main"
             :cwd nil
             :env '(("DEBUG" . "1"))
```

```
            :request "launch"
            :name "App:main"))

      (dap-register-debug-template
       "Poetry :: Run 'demo'"
       (list :type "poetry"
             :args "run demo"
             :cwd nil
             :env '(("DEBUG" . "1"))
             :request "launch"
             :name "App:demo"))

      (dap-register-debug-template
       "UV :: Run 'pytest'"
       (list :type "uv"
             :args "run pytest"
             :cwd nil
             :env '(("DEBUG" . "1"))
             :request "launch"
             :name "uv:pytest"))
      )

#+END_SRC

*** Lang: Treesitter
**** Lang: Treesitter.setup
#+NAME: lang-treesitter.setup
#+begin_src emacs-lisp

;; ---( treesitter-setup )-------------------------------------------------------------

(defun h7/treesitter-setup ()

  (setq treesit-language-source-alist
        '((ada "https://github.com/briot/tree-sitter-ada")
          (bash "https://github.com/tree-sitter/tree-sitter-bash")
          (cmake "https://github.com/uyha/tree-sitter-cmake")
          (css "https://github.com/tree-sitter/tree-sitter-css")
          (elisp "https://github.com/Wilfred/tree-sitter-elisp")
          (go "https://github.com/tree-sitter/tree-sitter-go")
          (html "https://github.com/tree-sitter/tree-sitter-html")
          (javascript "https://github.com/tree-sitter/tree-sitter-javascript"
                      "master" "src")
          (json "https://github.com/tree-sitter/tree-sitter-json")
          (make "https://github.com/alemuller/tree-sitter-make")
```

```
          (markdown "https://github.com/ikatyang/tree-sitter-markdown")
          (python "https://github.com/tree-sitter/tree-sitter-python")
          (toml "https://github.com/tree-sitter/tree-sitter-toml")
          (tsx "https://github.com/tree-sitter/tree-sitter-typescript"
               "master" "tsx/src")
          (typescript "https://github.com/tree-sitter/tree-sitter-typescript"
                      "master" "typescript/src")
          (yaml "https://github.com/ikatyang/tree-sitter-yaml")))

  ;; (mapc #'treesit-install-language-grammar
  ;;       (mapcar #'car treesit-language-source-alist))

  (setq major-mode-remap-alist
        '( ;;       (ada-mode . ada-ts-mode)
          ;;       (yaml-mode . yaml-ts-mode)
          (toml-mode . toml-ts-mode)
          ;;       (bash-mode . bash-ts-mode)
          ;;       (sh-mode . bash-ts-mode)
          ;;       (js2-mode . js-ts-mode)
          ;;       (typescript-mode . typescript-ts-mode)
          ;;       (conf-colon-mode . json-ts-mode)
          ;;       (json-mode . json-ts-mode)
          ;;       (css-mode . css-ts-mode)
          ;;       (python-mode . python-ts-mode)
          ))
  )

#+END_SRC


*** Lang: R
**** Lang: R/ess
#+NAME: lang-r.ess
#+begin_src emacs-lisp

  ;; ---( R )-------------------------------------------------------------

  (use-package ess
;;   :if (version≤ "25.1" emacs-version)
;;  :defer t
    :ensure t

    ;;:load-path "site-lisp/ess/lisp/"
    ;;:config (ess-toggle-underscore nil)
    :init
```

```
(add-hook 'ess-mode-hook
          (lambda ()

            (ess-set-style 'RStudio)

            ;; Replace \C-c with \M-c for CUA and ctrl key swap

            ;; ;; By popular demand:
            ;;(define-key map "\C-m"             'ess-newline-and-indent); = [RETURN]
            ;;(define-key map [remap yank]       'ess-yank)

            (define-key ess-mode-map (kbd "M-c M-c")      'ess-eval-region-and-go)

            (define-key ess-mode-map (kbd "M-c C-r")      'ess-eval-region)
            (define-key ess-mode-map (kbd "M-c M-r")      'ess-eval-region-and-go)
            (define-key ess-mode-map (kbd "M-c C-b")      'ess-eval-buffer)
            (define-key ess-mode-map (kbd "M-c M-b")      'ess-eval-buffer-and-go)
            (define-key ess-mode-map (kbd "M-c C-<up>")   'ess-eval-buffer-from-beg-to-here)
            (define-key ess-mode-map (kbd "M-c C-<down>") 'ess-eval-buffer-from-here-to-end)
            (define-key ess-mode-map (kbd "M-c C-f")      'ess-eval-function)
            (define-key ess-mode-map (kbd "M-c M-f")      'ess-eval-function-and-go)
            (define-key ess-mode-map (kbd "M-c C-c")      'ess-eval-region-or-function-or-pa
            (define-key ess-mode-map (kbd "M-c C-p")      'ess-eval-paragraph-and-step)
            (define-key ess-mode-map (kbd "M-c M-p")      'ess-eval-paragraph-and-go)
            (define-key ess-mode-map (kbd "M-c M-x")      'ess-eval-region-or-function-or-pa
            (define-key ess-mode-map (kbd "M-c M-n")      'ess-eval-line-and-step)
            (define-key ess-mode-map (kbd "M-c M-j")      'ess-eval-line)
            (define-key ess-mode-map [(control return)]   'ess-eval-region-or-line-and-step)
            (define-key ess-mode-map (kbd "M-c M-j")      'ess-eval-line-and-go)
            ;; the next three can only work in S/R - mode {FIXME}
            (define-key ess-mode-map (kbd "M-c M-a")      'ess-goto-beginning-of-function-or
            (define-key ess-mode-map (kbd "M-c M-e")      'ess-goto-end-of-function-or-para)
            (define-key ess-mode-map "\C-xnd"             'ess-narrow-to-defun-or-para)
            (define-key ess-mode-map "\C-xnf"             'ess-narrow-to-defun-or-para)
            (define-key ess-mode-map (kbd "M-c M-y")      'ess-switch-to-ESS-deprecated)
            (define-key ess-mode-map (kbd "M-c M-z")      'ess-switch-to-inferior-or-script-
            (define-key ess-mode-map (kbd "M-c C-z")      'ess-switch-to-inferior-or-script-
            (define-key ess-mode-map (kbd "C-c C-z")      'ess-switch-to-inferior-or-script-
            (define-key ess-mode-map (kbd "C-c M-l")      'ess-load-file)
            (define-key ess-mode-map (kbd "M-c M-l")      'ess-load-file); alias, as in 'iES
            (define-key ess-mode-map (kbd "M-c M-v")      'ess-display-help-on-object)
            ;;(define-key ess-mode-map "\C-c5\C-d"'ess-dump-object-into-edit-buffer-other-fr
            (define-key ess-mode-map (kbd "M-c M-s")      'ess-switch-process) ; use a

            ;; different process for the buffer.
```

```
                ;; (define-key map "\C-c\C-t"        'ess-execute-in-tb)
                ;;(define-key ess-mode-map (kbd "M-c \t")     'ess-complete-object-name-deprecat
                ;; (define-key ess-mode-map "\C-c\t"        'comint-dynamic-complete-filename)

                (unless (and (featurep 'emacs) (≥ emacs-major-version 24))
                  (define-key ess-mode-map (kbd "M-c <tab>")  'comint-dynamic-complete))
                (define-key ess-mode-map (kbd "M-c .")        'ess-list-object-completions)

                ;; wrong here (define-key ess-mode-map "\C-c\C-k" 'ess-request-a-process)
                (define-key ess-mode-map (kbd "M-c M-k")      'ess-force-buffer-current)
                (define-key ess-mode-map (kbd "M-c `")        'ess-show-traceback)
                (define-key ess-mode-map (kbd "M-c \\")       'ess-show-call-stack)

                ;;(define-key ess-mode-map (kbd "M-c .")      (lambda () (interactive) (message

                ;;(define-key ess-mode-map "{"                'ess-electric-brace)
                ;;(define-key ess-mode-map "}"                'ess-electric-brace)

                (define-key ess-mode-map (kbd "M-c M-q")      'ess-indent-exp)
                (define-key ess-mode-map (kbd "<M-S-right>")  'ess-mark-function-or-para)
                (if (featurep 'xemacs) ;; work around Xemacs bug (\C-\M-h redefines M-BS):
                    (define-key ess-mode-map [(meta backspace)] 'backward-kill-word))
                ;;(define-key ess-mode-map [delete]           'backward-delete-char-untabify)

                ;;(define-key ess-mode-map "\t"               'ess-indent-or-complete)
                (define-key ess-mode-map (kbd "M-c C-q")      'ess-quit)
                (define-key ess-mode-map (kbd "M-c M-r")      'ess-use-this-dir)

                ;; smart operators; most likely will go in the future into a separate local map
                ;;(define-key map ","          'ess-smart-comma)

                (define-key ess-mode-map (kbd "M-c M-d")       'ess-doc-map)
                (define-key ess-mode-map (kbd "M-c M-e")       'ess-extra-map)
                (define-key ess-mode-map (kbd "M-c M-t")       'ess-dev-map)
                (define-key ess-mode-map (kbd "M-c C-d")       'ess-doc-map)
                (define-key ess-mode-map (kbd "M-c C-e")       'ess-extra-map)
                (define-key ess-mode-map (kbd "M-c C-t")       'ess-dev-map)


;;              (ess-toggle-underscore nil))
                 ;; (define-key ess-mode-map (kbd "M-c M-c")
                 ;;   'ess-eval-region-and-go)
              ))
  (add-hook 'inferior-ess-mode-hook
              #'(lambda nil
```

```
                 (define-key inferior-ess-mode-map [\C-up]
                   'comint-previous-matching-input-from-input)
                 (define-key inferior-ess-mode-map [\C-down]
                   'comint-next-matching-input-from-input)
                 (define-key inferior-ess-mode-map [\C-x \t]
                   'comint-dynamic-complete-filename)
                 )
               )
    :commands R)

  (use-package ess-R-data-view
    :defer t)

  (use-package ess-R-object-popup
    :defer t)

  (use-package ess-R-data-smart-equals
    :disabled t)

  (use-package ess-R-data-smart-underscore
    :disabled t)


#+END_SRC

**** Lang: R/polymode
#+NAME: lang-r.ess.polymode
#+begin_src emacs-lisp

  ;; ---( polymode )-----------------------------------------------------------


(use-package polymode
  :ensure t
  :commands (poly-markdown+r-mode)
  :mode ((("\\.rmd\\'" . poly-markdown+r-mode)
     ("\\.Rmd\\'" . poly-markdown+r-mode))

  :init
  (autoload 'r-mode "ess-site.el" "Major mode for editing R source." t)
)


(use-package poly-markdown
  :ensure t
```

```
  :mode (
     ("\\.md" . poly-markdown-mode)
  )
)

(use-package poly-R
  :ensure t
)


#+END_SRC

*** Lang: Python
**** Lang: Python/mode
#+NAME: lang-python.mode
#+begin_src emacs-lisp


  ;; ---( python )-------------------------------------------------------------

  ;; @see: https://gitlab.com/nathanfurnal/dotemacs/-/snippets/2060535?utm_source=pocket_mylist
  ;; @see: https://github.com/jidicula/dotfiles/blob/main/init.el?utm_source=pocket_mylist


  ;; Built-in Python utilities
  (use-package python
    :ensure t
    :config
    ;; Remove guess indent python message
    (setq python-indent-guess-indent-offset-verbose nil)
    ;; Use IPython when available or fall back to regular Python
    (cond
     ((executable-find "ipython")
      (progn
        (setq python-shell-buffer-name "IPython")
        (setq python-shell-interpreter "ipython")
        (setq python-shell-interpreter-args "-i --simple-prompt")))
     ((executable-find "python3")
      (setq python-shell-interpreter "python3"))
     ((executable-find "python2")
      (setq python-shell-interpreter "python2"))
     (t
      (setq python-shell-interpreter "python"))))
```

```
;; Hide the modeline for inferior python processes
(use-package inferior-python-mode
  :ensure nil
  :hook (inferior-python-mode . hide-mode-line-mode))

;; Required to hide the modeline
(use-package hide-mode-line
  :ensure t
  :defer t)



;; (use-package python-mode
;;   :mode ("\\.py\\'" . python-mode)
;;   :interpreter ("python" . python-mode)
;;   :config
;;   (defvar python-mode-initialized nil)
;;   (defun my-python-mode-hook ()
;;     (unless python-mode-initialized
;;       (setq python-mode-initialized t)
;;       (info-lookup-add-help
;;        :mode 'python-mode
;;        :regexp "[a-zA-Z_0-9.]+"
;;        :doc-spec
;;        '(("(python)Python Module Index" )
;;          ("(python)Index"
;;           (lambda
;;             (item)
;;             (cond
;;              ((string-match
;;                "\\([A-Za-z0-9_]+\\)() (in module \\([A-Za-z0-9_.]+\\))" item)
;;               (format "%s.%s" (match-string 2 item)
;;                       (match-string 1 item))))))))
;;     (setq indent-tabs-mode nil)
;;     (set (make-local-variable 'parens-require-spaces) nil)
;;     (setq indent-tabs-mode nil)
;;     (bind-key "C-c C-z" 'python-shell python-mode-map)
;;     (unbind-key "C-c c" python-mode-map))
;;   (add-hook 'python-mode-hook 'my-python-mode-hook))
```

```
#+END_SRC
**** Lang: Python/env
```

```
#+NAME: lang-python.env
#+begin_src emacs-lisp

  ;; ---( virtual env )-------------------------------------------------------------

  (use-package with-venv
    :ensure t)

  ;; ---( pyvenv )----------------------------------------------------------------

  ;; Required to easily switch virtual envs
  ;; via the menu bar or with `pyvenv-workon`
  ;; Setting the `WORKON_HOME` environment variable points
  ;; at where the envs are located. I use (miniconda ^H) poetry.
  (use-package pyvenv
    :ensure t
    :defer t
    :config
    ;; Setting work on to easily switch between environments
    ;;(setenv "WORKON_HOME" (expand-file-name "~/miniconda3/envs/"))
    (setenv "WORKON_HOME" (expand-file-name "~/.cache/pypoetry/virtualenvs"))
    ;; Display virtual envs in the menu bar
    (setq pyvenv-menu t)
    ;; Restart the python process when switching environments
    (add-hook 'pyvenv-post-activate-hooks (lambda ()
                                            (pyvenv-restart-python)))
    :hook (python-mode . pyvenv-mode))


  ;; ---( poetry )----------------------------------------------------------------

  ;; (use-package poetry
  ;;   :ensure t
  ;;   ;; :init
  ;;   ;; imperfect tracking strategy causes lags in builds
  ;;   ;; (setq poetry-tracking-strategy 'switch-buffer)
  ;;   :hook
  ;;   ;; activate poetry-tracking-mode when python-mode is active
  ;;   (python-mode . poetry-tracking-mode)
  ;;   )

  (use-package poetry
    :ensure t
    :config
    (add-hook 'poetry-tracking-mode-hook (lambda () (remove-hook 'post-command-hook 'poetry-trac
```

```
    (add-hook 'python-mode-hook 'poetry-track-virtualenv)
    (add-hook 'projectile-after-switch-project-hook 'poetry-track-virtualenv))


  ;; ---( uv )-------------------------------------------------------------

(use-package uv-mode
    :ensure t
    :hook (python-mode . uv-mode-auto-activate-hook))

  ;;t ---( pipenv )-------------------------------------------------------------
  ;;
  ;; (use-package pipenv
  ;;   :unless (version< emacs-version "25.1")
  ;;   :defer t
  ;;   ;; :ensure t
  ;;   :hook (python-mode . pipenv-mode)
  ;;   :init
  ;;   (setq
  ;;    pipenv-projectile-after-switch-function
  ;;    #'pipenv-projectile-after-switch-extended))


#+END_SRC
**** Lang: Python/lsp
#+NAME: lang-python.lsp
#+begin_src emacs-lisp

  ;; ---( lsp-pyright )-----------------------------------------------------------------

  ;; Language server for Python
  ;; Read the docs for the different variables set in the config.

  (use-package lsp-pyright
    :ensure t
    :defer t
    :custom
    (lsp-pyright-langserver-command "pyright") ;; or basedpyright
    (lsp-pyright-disable-language-service nil)
    (lsp-pyright-disable-organize-imports nil)
    (lsp-pyright-auto-import-completions t)
    (lsp-pyright-use-library-code-for-types t)
    ;;(lsp-pyright-venv-path "~/.cache/pypoetry/virtualenvs")
    :config
    ;;(setq lsp-clients-python-library-directories '("/usr/" "~/miniconda3/pkgs"))
```

```
    ;;(setq lsp-clients-python-library-directories '("/usr/" "~/miniconda3/pkgs"))
    ;; (setq lsp-pyright-disable-language-service nil
    ;;       lsp-pyright-disable-organize-imports nil
    ;;       lsp-pyright-auto-import-completions t
    ;;       lsp-pyright-use-library-code-for-types t
    ;;       ;;lsp-pyright-venv-path "~/miniconda3/envs")
    ;;       lsp-pyright-venv-path "~/.cache/pypoetry/virtualenvs")
    :hook ((python-mode . (lambda ()
                           (require 'lsp-pyright) (lsp-deferred))))
    )

#+END_SRC
**** Lang: Python/tools
#+NAME: lang-python.tools
#+begin_src emacs-lisp


  ;; ---( pytest )-------------------------------------------------------------


(use-package python-pytest
  :ensure t
 :custom
 (python-pytest-confirm t))

  ;; ---( yapfify )-------------------------------------------------------------

  ;; Format the python buffer following YAPF rules
  ;; There's also blacken if you like it better.
  (use-package yapfify
    :ensure t
    :defer t
    :hook (python-mode . yapf-mode))


  ;; ---( python-black )-------------------------------------------------------------

  (use-package python-black
    ;;:delight python-black-on-save-mode "⍈"
    :ensure t
    :hook
    (python-mode . python-black-on-save-mode)
    :init
    (put 'python-black-command 'safe-local-variable #'stringp)
    (put 'python-black-extra-args 'safe-local-variable #'stringp)
```

```
    (put 'python-black-on-save-mode 'safe-local-variable #'booleanp)
    )

  ;; ---( pyisort )-------------------------------------------------------------

  (use-package py-isort
    :ensure t
    :after python
    :hook ((python-mode . pyvenv-mode)
           (before-save . py-isort-before-save)))


#+END_SRC
**** Lang: Python/elpy
#+NAME: lang-python.elpy
#+begin_src emacs-lisp


  ;; ---( python: elpy )--------------------------------------------------------------

  (use-package elpy
    :disabled t
    :preface

    ;; @see: https://elpy.readthedocs.org/en/latest/
    ;; @see: https://github.com/jorgenschaefer/elpy
    ;; @see: https://youtu.be/0kuCeS-mfyc

    (defvar elpy-mode-map
      (let ((map (make-sparse-keymap)))
        ;; Alphabetical order to make it easier to find free C-c C-X
        ;; bindings in the future. Heh.

        ;; (define-key map (kbd "<backspace>") 'python-indent-dedent-line-backspace)
        ;; (define-key map (kbd "<backtab>")   'python-indent-dedent-line)

        ;; (define-key map (kbd "C-M-x")   'python-shell-send-defun)

        (define-key map (kbd "M-c <")   'python-indent-shift-left)
        (define-key map (kbd "M-c >")   'python-indent-shift-right)

        (define-key map (kbd "M-c RET") 'elpy-importmagic-add-import)
        (define-key map (kbd "M-c M-b") 'elpy-nav-expand-to-indentation)
        (define-key map (kbd "M-c M-c") 'elpy-shell-send-region-or-buffer)
        (define-key map (kbd "M-c M-d") 'elpy-doc)
```

```
(define-key map (kbd "M-c M-e") 'elpy-multiedit-python-symbol-at-point)
(define-key map (kbd "M-c M-f") 'elpy-find-file)
(define-key map (kbd "M-c M-n") 'elpy-flymake-next-error)
(define-key map (kbd "M-c M-o") 'elpy-occur-definitions)
(define-key map (kbd "M-c M-p") 'elpy-flymake-previous-error)
(define-key map (kbd "M-c M-s") 'elpy-rgrep-symbol)
(define-key map (kbd "M-c M-t") 'elpy-test)
(define-key map (kbd "M-c M-v") 'elpy-check)
(define-key map (kbd "M-c M-z") 'elpy-shell-switch-to-shell)
(define-key map (kbd "M-c M-r i") 'elpy-importmagic-fixup)
(define-key map (kbd "M-c M-r p") 'elpy-autopep8-fix-code)
(define-key map (kbd "M-c M-r r") 'elpy-refactor)

;; (define-key map (kbd "<S-return>") 'elpy-open-and-indent-line-below)
;; (define-key map (kbd "<C-S-return>") 'elpy-open-and-indent-line-above)

;; (define-key map (kbd "<C-return>") 'elpy-shell-send-current-statement)

;; (define-key map (kbd "<C-down>") 'elpy-nav-forward-block)
;; (define-key map (kbd "<C-up>") 'elpy-nav-backward-block)
;; (define-key map (kbd "<C-left>") 'elpy-nav-backward-indent)
;; (define-key map (kbd "<C-right>") 'elpy-nav-forward-indent)

;; (define-key map (kbd "<M-down>") 'elpy-nav-move-line-or-region-down)
;; (define-key map (kbd "<M-up>") 'elpy-nav-move-line-or-region-up)
;; (define-key map (kbd "<M-left>") 'elpy-nav-indent-shift-left)
;; (define-key map (kbd "<M-right>") 'elpy-nav-indent-shift-right)

;; (define-key map (kbd "M-.")     'elpy-goto-definition)
;; (define-key map (kbd "M-TAB")   'elpy-company-backend)

(define-key map (kbd "<C-S-return>") 'elpy-open-and-indent-line-below)
;;(define-key map (kbd "<C-S-return>") 'elpy-open-and-indent-line-above)

;;(define-key map (kbd "<C-return>") 'elpy-shell-send-current-statement)

(define-key map (kbd "<M-right>") 'elpy-nav-forward-block)
(define-key map (kbd "<M-left>") 'elpy-nav-backward-block)
;; (define-key map (kbd "<C-S-left>") 'elpy-nav-backward-indent)
;; (define-key map (kbd "<C-S-right>") 'elpy-nav-forward-indent)

;; (define-key map (kbd "<M-S-down>") 'elpy-nav-move-line-or-region-down)
;; (define-key map (kbd "<M-S-up>") 'elpy-nav-move-line-or-region-up)
(define-key map (kbd "<M-S-left>") 'elpy-nav-indent-shift-left)
(define-key map (kbd "<M-S-right>") 'elpy-nav-indent-shift-right)
```

```
        (define-key map [(meta prior)]    'elpy-goto-definition)
        (define-key map [(meta next)]     'pop-tag-mark)

        (define-key map [(control menu)]   'elpy-company-backend)

        map)
      "Key map for the Emacs Lisp Python Environment.")
    :config
    (elpy-enable)
    (setq python-shell-interpreter "jupyter"
          python-shell-interpreter-args "console --simple-prompt")

    ;; (elpy-use-ipython "ipython3")
    (defalias 'workon 'pyvenv-workon))

  (setenv "PYTHONIOENCODING" "utf-8")
  (add-to-list 'process-coding-system-alist '("python" . (utf-8 . utf-8)))
  (add-to-list 'process-coding-system-alist '("elpy" . (utf-8 . utf-8)))
  (add-to-list 'process-coding-system-alist '("flake8" . (utf-8 . utf-8)))

#+END_SRC
**** Lang: Python/ein
#+NAME: lang-python.ein
#+begin_src emacs-lisp

  ;; ---( python: ein )-------------------------------------------------------------


  (use-package ein
    :unless (version< emacs-version "25.1")
    ;; :defer t
    ;;:ensure t
    :disabled t
    :init
    (progn
      (with-eval-after-load 'ein-notebooklist
        (define-key ein:notebooklist-mode-map (kbd "<S-return>") 'ein:worksheet-execute-cell-and
        (define-key ein:notebooklist-mode-map (kbd "<C-return>") 'ein:worksheet-execute-cell)
        ))
    :config
    (defalias 'eip 'ein:notebooklist-open))
```

```
;; (use-package ein
;;   :unless (version< emacs-version "25.1")
;;   :ensure t
;;   :defer t
;;   :commands ein:notebooklist-open
;;   :init
;;   ;; (progn
;;   ;;   (with-eval-after-load 'ein-notebooklist
;;   ;;     ;; removing keybindings
;;   ;;     (define-key ein:notebook-mode-map (kbd "M-p") nil)
;;   ;;     (define-key ein:notebook-mode-map (kbd "<M-up>") nil)
;;   ;;     (define-key ein:notebook-mode-map (kbd "<M-down>") nil)
;;   ;;     ;; changing keybinding
;;   ;;     (define-key ein:notebook-mode-map (kbd "C-s") 'ein:notebook-save-notebook-command)
;;   ;;     (define-key ein:notebook-mode-map (kbd "<M-S-up>") 'ein:worksheet-move-cell-up)
;;   ;;     (define-key ein:notebook-mode-map (kbd "<M-S-down>") 'ein:worksheet-move-cell-down
;;   :config
;;   (defalias 'einp 'ein:notebooklist-open)
;;   (defalias 'eins 'ein:jupyter-server-start)
;;   )

;; ---( python: 0mq )-------------------------------------------------------------

;; @see: https://github.com/nnicandro/emacs-zmq
;; @see: https://github.com/nnicandro/emacs-zmq/issues/48
;; dnf install zeromq-devel
;; apt install libczmq-dev


;; python and jupyter
;;; custom zmq build - see https://github.com/alexmurray/emacs-snap/issues/66
;;; @see: https://github.com/martibosch/snakemacs/blob/main/main.el#L346

(cond ((getenv "EMACS_SNAP_DIR")

       (let* ((emacs-snap-dir (file-name-as-directory (getenv "EMACS_SNAP_DIR")))
              (process-environment (append process-environment `(,(concat "CC=" emacs-snap-dir
                                                                  ,(concat "CXX=" emacs-snap-di
                                                                  ,(concat "CFLAGS=--sysroot="
                                        ,(concat "CPPFLAGS=--sysroot=" emacs-snap-dir)
                                        ,(concat "LDFLAGS=--sysroot=" emacs-snap-dir " -L" emacs-
         (use-package zmq
          :if (h7/use-py-jupyter)
          :defer t
          :ensure t)
```

```
          ))
        (t
          (use-package zmq
            :if (h7/use-py-jupyter)
            :defer t
            :ensure t)
          ))



  ;; (use-package zmq
  ;;   :if (h7/use-py-jupyter)
  ;;   :defer t
  ;;   :ensure t
  ;;   :preface

;;     (package-install "zmq")
;;     (vterm)

;; cd ~/.emacs.d/elpa

;; ls -lda zmq*
;; cd      zmq*


;; ES=${EMACS_SNAP_DIR:-/snap/emacs/current}
;; export CC=${ES}/usr/bin/gcc-10
;; export CXX=${ES}/usr/bin/g++-10
;; export CFLAGS=--sysroot=${ES}
;; export LDFLAGS="--sysroot=${ES} -L${ES}/usr/lib"

;; printenv | grep -i -e ^cc= -e ^cxx= -e ^cflags= -e ^ldflags=
;; ls -l $CC $CXX

;; make all

;;     (package-install "jupyter")

    ;; :init
    ;; :config
    ;; )

  ;; (use-package zmq
  ;;   :ensure t
  ;;   :preface
```

```
;;   (when (getenv "EMACS_SNAP_DIR")
;;     (unless (directory-files-recursively (concat user-emacs-directory "") "zmq-.*\\.so$" ni
;;              (progn
;;                ;; @see: https://github.com/nnicandro/emacs-zmq/issues/48
;;                (let* ((emacs-snap-dir (file-name-as-directory (getenv "EMACS_SNAP_DIR")))
;;                       (process-environment
;;                        (append `(,(concat "CC=" emacs-snap-dir "usr/bin/gcc-10" )
;;                                  ,(concat "CXX=" emacs-snap-dir "usr/bin/g++-10")
;;                                  ,(concat "CFLAGS=--sysroot=" emacs-snap-dir " -B" emacs-sn
;;                                  ,(concat "CPATH=" (file-name-directory (car (file-expand-w
;;                    ,(concat "CPPFLAGS=--sysroot=" emacs-snap-dir)
;;                    ,(concat "LDFLAGS=--sysroot=" emacs-snap-dir " -L" emacs-snap-dir "usr/li
;;                                  ,(concat "PKG_CONFIG_PATH=" (car (file-expand-wildcards (c
;;                              process-environment)))
;;                   ;; @see: https://github.com/nnicandro/emacs-zmq/issues/48#issuecomment-220
;;                   (when (fboundp 'native-compile-async)
;;                     (progn
;;                       (setq native-comp-deferred-compilation t
;;                             native-comp-deferred-compilation-deny-list
;;                             '("/mu4e.*\\.el$" "jupyter" "zmq" "eaf" "eaf-mode" "emacs-zmq"))
;;                ;; (custom-set-variables
;;                ;;  '(native-comp-async-report-warnings-errors 'silent))
;;                ;; ;; (let ((snap (file-name-as-directory "/snap/emacs/current")))
;;                ;; ;;    (setq-default native-comp-driver-options (list (concat "--sysroot="
;;                ;; ;;                                                    (concat "-B" snap "usr

;;                   (load-library "zmq")

;;                   ))))
;;   :init
;;   :config
;;   )


;; ---( python: jupyter )-------------------------------------------------------------

;; @see: https://sqrtminusone.xyz/posts/2021-05-01-org-python/

(use-package jupyter
  :if (h7/use-py-jupyter)
  :defer t
  :ensure t
  :init
  :config
  )
```

```
;; ---( python: code cells )-------------------------------------------------------------

;; @see: https://github.com/martibosch/snakemacs/blob/main/main.el#L444

(use-package code-cells
  :ensure t
  :after org
  :config
  (setq code-cells-convert-ipynb-style '(("pandoc" "--to" "ipynb" "--from" "org")
                        ("pandoc" "--to" "org" "--from" "ipynb")
                        org-mode))
  ;; see https://github.com/astoff/code-cells.el/issues/22
  ;; (defun gm/jupyter-eval-region (beg end)
  ;;   (jupyter-eval-region nil beg end))
  ;; (add-to-list 'code-cells-eval-region-commands '(jupyter-repl-interaction-mode . gm/jupyte
  (let ((map code-cells-mode-map))
    (define-key map (kbd "C-c <up>") 'code-cells-backward-cell)
    (define-key map (kbd "C-c <down>") 'code-cells-forward-cell)
    (define-key map (kbd "M-<up>") 'code-cells-move-cell-up)
    (define-key map (kbd "M-<down>") 'code-cells-move-cell-down)
    (define-key map (kbd "C-c C-c") 'code-cells-eval)
    ;; Overriding other minor mode bindings requires some insistence...
    (define-key map [remap jupyter-eval-line-or-region] 'code-cells-eval)))
    (defun my/new-notebook (notebook-name &optional kernel)
      "Creates an empty notebook in the current directory with an associated kernel."
      (interactive "sEnter the notebook name: ")
      (when (file-name-extension notebook-name)
        (setq notebook-name (file-name-sans-extension notebook-name)))
      (unless kernel
        (setq kernel (jupyter-kernelspec-name (jupyter-completing-read-kernelspec))))
      (unless (executable-find "jupytext")
        (error "Can't find \"jupytext\""))
      (let ((notebook-py (concat notebook-name ".py")))
        (shell-command (concat "touch " notebook-py))
        (shell-command (concat "jupytext --set-kernel " kernel " " notebook-py))
        (shell-command (concat "jupytext --to notebook " notebook-py))
        (shell-command (concat "rm " notebook-py))
        (message (concat "Notebook successfully created at " notebook-name ".ipynb"))))
```

#+END_SRC

```
*** Lang: Julia
#+NAME: lang-julia
#+begin_src emacs-lisp

  ;; ---( julia )-------------------------------------------------------------

  ;; @see: https://github.com/JuliaEditorSupport/julia-emacs
  ;; @see: https://github.com/tpapp/julia-repl
  ;; @see: https://github.com/nnicandro/emacs-jupyter
  ;; @see: https://julia-users-zurich.github.io/talks/talk-2018-04/emacs.html
  ;; @see: https://github.com/cgroll/dot_emacs.d/blob/master/init.el

  (use-package julia-mode
     :ensure t
     :defer t
     :commands julia-mode
     :mode ("\\.jl$" . julia-mode)
     :init
     (progn
        (autoload 'julia-mode "julia-mode" nil t)
        (setq inferior-julia-program-name "julia")
        )
     :config
     (progn
        (setq inferior-julia-program-name "julia")
        )
     )


  (use-package julia-repl
     :ensure t
     :defer t
     :config
     (progn
       (add-to-list 'julia-mode-hook 'julia-repl-mode)
       )
     )

  ;; ;; allow julia to be loaded through call to julia-mode or
  ;; ;; ess-inferior process
  ;; ;; follow-ups: etags?
  ;; (use-package julia-mode
  ;;    :defer t
  ;;    :commands julia-mode
  ;;    :mode ("\\.jl$" . julia-mode)
```

```
  ;;    :init
  ;;    (progn
  ;;       (autoload 'julia-mode "julia-mode" nil t)
  ;;       (setq inferior-julia-program-name "/usr/bin/julia")
  ;;       )
  ;;    :config
  ;;    (progn
  ;;       (add-to-list 'julia-mode-hook 'cg/modify-current-syntax-table)
  ;;       (setq inferior-julia-program-name "/usr/bin/julia")
  ;;       (add-to-list 'julia-mode-hook 'cg/command-line-keybindings)
  ;;       ;; (add-to-list 'inferior-ess-mode-hook 'cg/command-line-keybindings)
  ;;       )
  ;;    )

  ;; (use-package ess-julia.el
  ;;    :defer t
  ;;    :commands julia
  ;;    :init                                ; run before actual loading
  ;;    (progn
  ;;       (autoload 'julia "ess-julia.el" nil t)
  ;;       (setq inferior-julia-program-name "/usr/bin/julia")
  ;;       )
  ;;    :config
  ;;    (progn
  ;;       (require 'ess-site)
  ;;       (setq inferior-julia-program-name "/usr/bin/julia")
  ;;       (setq ess-tracebug-prefix "\M-c")   ; define debug-mode starting key
  ;;       (setq ess-use-tracebug t)           ; tracebug is called for R
  ;;                                           ; AND JULIA!!
  ;;       (setq ess-tracebug-inject-source-p t)
  ;;       (add-to-list 'julia-mode-hook 'cg/command-line-keybindings)
  ;;       ;; (add-to-list 'inferior-ess-mode-hook 'cg/command-line-keybindings)
  ;;       )
  ;;    )
  ;; ;; in order to add ess-process afterward, apply julia-mode again on
  ;; ;; open buffers - probably ess-julia.el has to be loaded again also:
  ;; ;; M-x load-file ess-julia.el

#+END_SRC


*** Lang/end
#+NAME: lang-end
#+begin_src emacs-lisp
```

```
  ;; }}}  .lang

#+END_SRC


** Keys
*** keys/develop

#+begin_src emacs-lisp

;; ============================================
;; ---( Function Keys )-----
;; ============================================
(message "SITE:K-FUNKEYS")


;; ---( F1: Help )-------------------------------------------------------


(global-set-key [f1] 'help )
;; (global-set-key [(shift f1)] 'woman )
(global-set-key [(control f1)] 'find-function )
(global-set-key [(meta f1)] 'function-key-error)
(global-set-key [(shift meta f1)] 'function-key-error)


;; ---( F2: Bookmarks/Breakpoints )-------------------------------------

(global-set-key [f2] 'dashboard-open)
(global-set-key [(shift f2)] 'bookmark-set )

;; (global-set-key [(control f2)]
;;     #'(lambda () (interactive)
;;        (if (eq hs-minor-mode nil)
;;     (progn
;;       (hs-minor-mode t)
;;       (hs-hide-all))
;;          (progn
;;            (hs-toggle-hiding)
;;     (hs-minor-mode nil)))))

;; (global-set-key [(control f2)]
;;     #'(lambda () (interactive)
;;          (progn
;;            ;;(vimish-fold-delete-all)
```

```
;;            (vimish-fold-from-marks)
;;     (vimish-fold-toggle))))
;; ;; (global-set-key [(shift control f2)]
;; ;;     #'(lambda () (interactive)
;; ;;          (progn
;; ;;            (vimish-fold-from-marks)
;; ;;            ;; (outline-hide-sublevels 1)
;; ;;      (vimish-fold-refold-all))))

(global-set-key [(control f2)]
    #'(lambda () (interactive)
         (progn
           (outline-hide-sublevels 1))))

(global-set-key [(shift control f2)]
    #'(lambda () (interactive)
         (progn
           (outline-show-all))))

(global-set-key [(meta f2)] 'bookmark-bmenu-list)
(global-set-key [(hyper f2)] 'bookmark-bmenu-list)
(global-set-key [(shift meta f2)]
    #'(lambda () (interactive)
        (progn
          (toggle-line-wrapping)
          (linum-mode 'toggle))))


;; ---( F3: ISearch/Find )-----------------------------------------------------

(global-set-key [f3] 'isearch-repeat-forward )
(global-set-key [(shift f3)] 'isearch-repeat-backward )
(global-set-key [(control f3)] 'isearch-iforward )
(global-set-key [(meta f3)] 'occur )
(global-set-key [(shift meta f3)] 'function-key-error)

;; ---( F4: Fold )-----------------------------------------------------

(global-set-key [f4] 'vimish-fold-toggle) ;; or 'vimish-fold is on selected regzion
(global-set-key [(shift f4)] 'hydra-fold/body)
(global-set-key [(control f4)] 'vimish-fold-refold-all-from-marks)
(global-set-key [(shift control f4)] 'vimish-fold-unfold-all)
(global-set-key [(shift meta f4)] 'vimish-fold)
;; (global-set-key [f4] 'call-last-kbd-macro)
;; (global-set-key [(shift f4)] 'start-or-end-kbd-macro )
```

```
;; (global-set-key [(control f4)] 'edit-last-kbd-macro )
;; (global-set-key [(meta f4)] 'kbd-macro-query )
;; (global-set-key [(shift meta f4)] 'edit-last-kbd-macro )
;; ;; (global-set-key [(control f4)] 'start-kbd-macro )
;; ;; (global-set-key [(meta f4)] 'end-kbd-macro )

;; ---( F5: Search/Grep )-----------------------------------------------------

(global-set-key [f5] 'isearch-forward-regexp )
(global-set-key [(shift f5)] 'isearch-backward-regexp )
(global-set-key [(control f5)] 'find-grep-dired )
(global-set-key [(meta f5)] 'grep  )
(global-set-key [(shift meta f5)] 'function-key-error)

;; ---( F6: Replace/Ediff )-------------------------------------------------------

(global-set-key [f6] 'query-replace )
(global-set-key [(shift f6)] 'query-replace-regexp )
(global-set-key [(control f6)] 'compare-windows )
(global-set-key [(meta f6)] 'ediff )
(global-set-key [(shift meta f6)] 'function-key-error)

;; ---( F7: Debug/Step )-----------------------------------------------------

;; (global-set-key [f7] 'gud-step ) ;;@TODO: move to local mode map
;; ;; (global-set-key [(control f7)] 'function-key-error)
;; (global-set-key [(meta f7)] 'function-key-error)
;; (global-set-key [(shift f7)] 'function-key-error)
;; (global-set-key [(shift meta f7)] 'function-key-error)

;; ---( F8: Debug/Next )-----------------------------------------------------

;; ;;(global-set-key [f8] 'function-key-error ) ;;WM expose
;; (global-set-key [(control f8)] 'gud-next) ;;@TODO: move to local mode map
;; (global-set-key [(meta f8)] 'function-key-error)
;; (global-set-key [(shift f8)] 'function-key-error)
;; (global-set-key [(shift meta f8)] 'function-key-error)

;; ---( F9: compile/run )-----------------------------------------------------

;;(global-set-key [f9] 'function-key-error ) ;;WM expose
;;(global-set-key [f9] 'perldb ) ;;@TODO: move to local mode map
(global-set-key [(f9)] 'eshell-toggle )
(global-set-key [(shift f9)] 'eshell-here )
```

```
(cond
 ((fboundp 'vterm);;
  (progn
    (message "vterm:bind [C-u] [C-u] C-F9")
    (global-set-key [(control f9)] 'vterm-here )
    ))
 ((fboundp 'multi-term);;
  (progn
    (global-set-key [(control f9)] 'multi-term )
    ))
 (t ;; fallback to VC bindings
  (progn
    (global-set-key [(control f9)] 'ansi-term )
    ))
 )

(global-set-key [(shift meta f9)] 'projectile-run-vterm )

;;(global-set-key [(shift meta f9)] 'mode-compile-kill )
;;(global-set-key [(meta f9)] 'mode-compile )
;;(global-set-key [(meta f9)] 'recompile )
;;(global-set-key [(shift meta f9)] 'compile)
;;(global-set-key [(meta f9)] 'shell)
(global-set-key [(meta f9)] 'list-processes)


;; ---( F10: UI )-----------------------------------------------------

;;(global-set-key [f10] 'menu )
;;(global-set-key [(control f10)] 'menu-bar-mode )
(global-set-key [(control f10)] 'toggle-menubar )
(global-set-key [(shift f10)] 'toggle-toolbar )
(global-set-key [(meta f10)] 'speedbar )
(global-set-key [(hyper f10)] 'treemacs )
(global-set-key [(shift meta f10)] 'toggle-gutter)


;; ---( F11: VCS )----------------------------------------------------

(cond
 ((fboundp 'magit-status);; Git magit
  (progn
    (global-set-key [f11] 'magit-status )
    (global-set-key [(shift meta f11)] 'vc-next-action)
    (global-set-key [(meta f11)] 'vc-diff )
```

```
    (global-set-key [(control meta f11)] 'toggle-fullscreen )
    ))
 (t ;; fallback to VC bindings
  (progn
    (global-set-key [f11] 'vc-next-action )
    (global-set-key [(shift f11)] 'vc-annotate )
    (global-set-key [(control f11)] 'vc-directory )
    (global-set-key [(meta f11)] 'vc-version-diff )
    (global-set-key [(shift meta f11)] 'function-key-error)
    (global-set-key [(control meta f11)] 'toggle-fullscreen )
    ))
 )




;;(define-key speedbar-key-map [button1] 'dframe-click)


;; ---( F12: Frames )-----------------------------------------------------

;; (global-set-key [f12] 'make-frame )
(global-set-key [f12] 'h7/make-note-frame )
(global-set-key [(control f12)] 'delete-frame )
(global-set-key [(shift f12)] 'buffer-menu )
(global-set-key [(meta f12)] 'toggle-frame-maximized )
(global-set-key [(shift meta f12)] 'function-key-error)




#+END_SRC
```

Next Configuration (TOBE)

```
** Lang
*** Lang/begin
#+NAME: lang-begin
#+begin_src emacs-lisp

  ;; ;;;/////////////////////////////////////////////////////////////////
  ;; {{{  @LANG
```

```
  ;; ;;;/////////////////////////////////////////////////////////////

#+END_SRC




*** Lang: LSP
**** Lang: LSP.setup
#+NAME: lang-lsp.setup
#+begin_src emacs-lisp

  ;; ---( lsp-setup )-------------------------------------------------------------

(defun h7/lsp-setup ()
    (interactive)

  ;;lsp server install

  ;; @see
  (message "https://emacs-lsp.github.io/lsp-mode/page/languages/")

  ;; python
  (lsp-install-server t 'pyright)

    ;; html
  (lsp-install-server t 'html-ls)
  ;; css
  (lsp-install-server t 'css-ls)
  ;; json
  (lsp-install-server t 'json-ls)
    ;; dockerfile
  (lsp-install-server t 'dockerfile-ls)
  ;; bash
  (lsp-install-server t 'bash-ls)
  ;; ansible
  (lsp-install-server t 'ansible-ls)
  ;; yaml
  (lsp-install-server t 'yamlls)

  ;; graphql - postgres dependency
  ;; (lsp-install-server t 'graphql-ls)
  ;; powershell - pwsh.exe requisite
  ;; (lsp-install-server t 'pwsh-ls)
```

```
  ;; prolog

  (message "swipl -g 'pack_install(lsp_server).")
  )


#+END_SRC

**** Lang: LSP.mode
#+NAME: lang-lsp.mode
#+begin_src emacs-lisp

;; ---( flycheck )-------------------------------------------------------------

(use-package flycheck
  :ensure t
  :init (global-flycheck-mode)
  )

;; ---( LSP mode )------------------------------------------------------------

(use-package lsp-mode
  :ensure t
  :init
  ;; "C-l" shadows recenter-top-bottom; use the standard lsp prefix.
  ;;(setq lsp-keymap-prefix "C-c l")
  (setq lsp-keymap-prefix "C-l")
  :hook (
         (python-mode . lsp-deferred)
         (python-ts-mode . lsp-deferred)
         (lsp-mode . lsp-enable-which-key-integration)
         )
  :commands (lsp lsp-deferred)
  :custom
  ;; Performance: increase IPC read buffer (critical for large PyTorch stubs).
  (read-process-output-max (* 1024 1024))
  ;; Use flycheck (already running globally) as the diagnostics provider.
  (lsp-diagnostics-provider :flycheck)
  ;; Disable pylsp/pyls; pyright (basedpyright) is the sole Python server.
  (lsp-disabled-clients '(pyls pylsp))
  ;; Snippets require yasnippet which is currently disabled.
  (lsp-enable-snippet nil)
  ;; Breadcrumb is useful for NLP codebases with deep class hierarchies.
  (lsp-headerline-breadcrumb-enable t)
  (lsp-headerline-breadcrumb-segments '(project file symbols))
```

```
  ;; Lens costs a round-trip per buffer; disable by default.
  (lsp-lens-enable nil)
  ;; Idle delay: 0.3 s gives responsive feedback on fast machines.
  (lsp-idle-delay 0.3)
  :config
  (dolist (dir '(
                 "[/\\\\]\\.cache"
                 "[/\\\\]\\.mypy_cache"
                 "[/\\\\]\\.pytest_cache"
                 "[/\\\\]\\.Rproj.user"
                 "[/\\\\]venv$"
                 "[/\\\\]build$"
                 "[/\\\\]dist$"
                 "[/\\\\]docker$"
                 "[/\\\\]notes$"
                 "[/\\\\]data$"
                 "[/\\\\]home$"
                 "[/\\\\]logs$"
                 "[/\\\\]renv$"
                 "[/\\\\]temp$"
                 "[/\\\\]_targets"
                 ))
    (push dir lsp-file-watch-ignored-directories))
  )

(use-package lsp-ui
  :ensure t
  :after lsp
  :hook (lsp-mode . lsp-ui-mode)
  :bind (:map lsp-ui-mode-map
              ("C-c i" . lsp-ui-imenu))
  :custom
  (lsp-ui-doc-position 'at-point)
  (lsp-ui-doc-enable t)
  ;; Show on demand only (M-. or hover); avoid constant pop-up noise.
  (lsp-ui-doc-show-with-cursor nil)
  (lsp-ui-sideline-enable t)
  (lsp-ui-sideline-show-diagnostics t)
  (lsp-ui-sideline-show-code-actions nil)
  (lsp-ui-imenu-enable t)
  (lsp-ui-flycheck-enable t)
  (lsp-ui-doc-delay 1.5)
  )
```

```
(use-package consult-lsp
  :ensure t
  :defer t
  :after lsp-mode
  :commands (consult-lsp-diagnostics consult-lsp-symbols consult-lsp-file-symbols)
  )

;; (use-package company-lsp
;;   :ensure t
;;   :defer t
;;   :after lsp
;;   :commands company-lsp
;;   )


(use-package lsp-treemacs
  :ensure t
  :defer t
  :after lsp-mode
  :commands lsp-treemacs-errors-list
  )


#+END_SRC
**** Lang: LSP.dap
#+NAME: lang-lsp.mode.dap
#+begin_src emacs-lisp

;; ---( dap )----------------------------------------------------------------

(use-package dap-mode
  :ensure t
  :after lsp-mode
  :commands dap-debug
  :hook (

         (python-mode . dap-mode)
         (python-mode . dap-ui-mode)
         (python-ts-mode . dap-mode)
         (python-ts-mode . dap-ui-mode)
         (dap-stopped . (lambda (arg) (call-interactively #'dap-hydra)))
         )
  :custom
  (lsp-enable-dap-auto-configure t)
  ;; (dap-auto-configure-features '(sessions locals controls tooltip))
```

```
  :config
  ;; (dap-auto-configure-mode)
  (require 'dap-hydra)
  (require 'dap-python)
  (setq dap-python-debugger 'debugpy)

  ;; Use pet to find the project's interpreter rather than with-venv.
  (defun dap-python--pyenv-executabl-find (command)
    (or (and (fboundp 'pet-executable-find)
             (pet-executable-find command))
        (with-venv (executable-find command))))


  (dap-register-debug-template
   "UV :: Run 'main'"
   (list :type "uv"
         :args "run main"
         :cwd nil
         :env '(("DEBUG" . "1"))
         :request "launch"
         :name "App:main"))

  (dap-register-debug-template
   "UV :: Run 'demo'"
   (list :type "uv"
         :args "run demo"
         :cwd nil
         :env '(("DEBUG" . "1"))
         :request "launch"
         :name "App:demo"))

  (dap-register-debug-template
   "UV :: Run 'pytest'"
   (list :type "uv"
         :args "run pytest"
         :cwd nil
         :env '(("DEBUG" . "1"))
         :request "launch"
         :name "uv:pytest"))
  )

#+END_SRC

*** Lang: Treesitter
**** Lang: Treesitter.setup
```

```
#+NAME: lang-treesitter.setup
#+begin_src emacs-lisp

;; ---( treesitter-setup )-------------------------------------------------------------

(defun h7/treesitter-setup ()

  ;; treesit-auto manages grammar installation and major-mode-remap-alist
  ;; automatically. It supersedes the manual h7/treesitter-setup function.
  ;;
  ;; To install a grammar interactively: M-x treesit-install-language-grammar
  ;; treesit-auto-install 'prompt asks before compiling any grammar.
  (setq treesit-language-source-alist
        '((ada "https://github.com/briot/tree-sitter-ada")
          (bash "https://github.com/tree-sitter/tree-sitter-bash")
          (cmake "https://github.com/uyha/tree-sitter-cmake")
          (css "https://github.com/tree-sitter/tree-sitter-css")
          (elisp "https://github.com/Wilfred/tree-sitter-elisp")
          (go "https://github.com/tree-sitter/tree-sitter-go")
          (html "https://github.com/tree-sitter/tree-sitter-html")
          (javascript "https://github.com/tree-sitter/tree-sitter-javascript"
                      "master" "src")
          (json "https://github.com/tree-sitter/tree-sitter-json")
          (make "https://github.com/alemuller/tree-sitter-make")
          (markdown "https://github.com/ikatyang/tree-sitter-markdown")
          (python "https://github.com/tree-sitter/tree-sitter-python")
          (toml "https://github.com/tree-sitter/tree-sitter-toml")
          (tsx "https://github.com/tree-sitter/tree-sitter-typescript"
               "master" "tsx/src")
          (typescript "https://github.com/tree-sitter/tree-sitter-typescript"
                      "master" "typescript/src")
          (yaml "https://github.com/ikatyang/tree-sitter-yaml")))
  )


  (use-package treesit-auto
    :ensure t
    :custom
    ;; Ask before downloading and compiling a grammar.
    ;; Change to t for silent auto-install (useful on CI or container first-boot).
    (treesit-auto-install 'prompt)
    :config
    ;; Register all grammars in auto-mode-alist and populate major-mode-remap-alist.
    ;; This activates python-ts-mode automatically when the python grammar is present.
    (treesit-auto-add-to-auto-mode-alist 'all)
```

```
    (global-treesit-auto-mode))
  ;; NOTE: h7/treesitter-setup is retained below as a convenience utility for
  ;; bulk-installing all grammars at once (e.g. on a fresh machine).
  ;; Call it manually: M-x h7/treesitter-setup
  (defun h7/treesitter-setup ()
    "Install all tree-sitter grammars defined in `treesit-language-source-alist'."
    (interactive)
    (mapc #'treesit-install-language-grammar
          (mapcar #'car treesit-language-source-alist))


  (setq major-mode-remap-alist
        '( ;;      (ada-mode . ada-ts-mode)
          ;;      (yaml-mode . yaml-ts-mode)
          (toml-mode . toml-ts-mode)
          ;;      (bash-mode . bash-ts-mode)
          ;;      (sh-mode . bash-ts-mode)
          ;;      (js2-mode . js-ts-mode)
          ;;      (typescript-mode . typescript-ts-mode)
          ;;      (conf-colon-mode . json-ts-mode)
          ;;      (json-mode . json-ts-mode)
          ;;      (css-mode . css-ts-mode)
          ;;      (python-mode . python-ts-mode)
          ))
  )

#+END_SRC



*** Lang: R
**** Lang: R/ess
#+NAME: lang-r.ess
#+begin_src emacs-lisp

  ;; ---( R )-------------------------------------------------------------

  (use-package ess
;;   :if (version≤ "25.1" emacs-version)
;;  :defer t
    :ensure t

    ;;:load-path "site-lisp/ess/lisp/"
    ;;:config (ess-toggle-underscore nil)
    :init
```

```
(add-hook 'ess-mode-hook
          (lambda ()

            (ess-set-style 'RStudio)

            ;; Replace \C-c with \M-c for CUA and ctrl key swap

            ;; ;; By popular demand:
            ;;(define-key map "\C-m"             'ess-newline-and-indent); = [RETURN]
            ;;(define-key map [remap yank]       'ess-yank)

            (define-key ess-mode-map (kbd "M-c M-c")      'ess-eval-region-and-go)

            (define-key ess-mode-map (kbd "M-c C-r")      'ess-eval-region)
            (define-key ess-mode-map (kbd "M-c M-r")      'ess-eval-region-and-go)
            (define-key ess-mode-map (kbd "M-c C-b")      'ess-eval-buffer)
            (define-key ess-mode-map (kbd "M-c M-b")      'ess-eval-buffer-and-go)
            (define-key ess-mode-map (kbd "M-c C-<up>")   'ess-eval-buffer-from-beg-to-here)
            (define-key ess-mode-map (kbd "M-c C-<down>") 'ess-eval-buffer-from-here-to-end)
            (define-key ess-mode-map (kbd "M-c C-f")      'ess-eval-function)
            (define-key ess-mode-map (kbd "M-c M-f")      'ess-eval-function-and-go)
            (define-key ess-mode-map (kbd "M-c C-c")      'ess-eval-region-or-function-or-pa
            (define-key ess-mode-map (kbd "M-c C-p")      'ess-eval-paragraph-and-step)
            (define-key ess-mode-map (kbd "M-c M-p")      'ess-eval-paragraph-and-go)
            (define-key ess-mode-map (kbd "M-c M-x")      'ess-eval-region-or-function-or-pa
            (define-key ess-mode-map (kbd "M-c M-n")      'ess-eval-line-and-step)
            (define-key ess-mode-map (kbd "M-c M-j")      'ess-eval-line)
            (define-key ess-mode-map [(control return)]   'ess-eval-region-or-line-and-step)
            (define-key ess-mode-map (kbd "M-c M-j")      'ess-eval-line-and-go)
            ;; the next three can only work in S/R - mode {FIXME}
            (define-key ess-mode-map (kbd "M-c M-a")      'ess-goto-beginning-of-function-or
            (define-key ess-mode-map (kbd "M-c M-e")      'ess-goto-end-of-function-or-para)
            (define-key ess-mode-map "\C-xnd"             'ess-narrow-to-defun-or-para)
            (define-key ess-mode-map "\C-xnf"             'ess-narrow-to-defun-or-para)
            (define-key ess-mode-map (kbd "M-c M-y")      'ess-switch-to-ESS-deprecated)
            (define-key ess-mode-map (kbd "M-c M-z")      'ess-switch-to-inferior-or-script-
            (define-key ess-mode-map (kbd "M-c C-z")      'ess-switch-to-inferior-or-script-
            (define-key ess-mode-map (kbd "C-c C-z")      'ess-switch-to-inferior-or-script-
            (define-key ess-mode-map (kbd "C-c M-l")      'ess-load-file)
            (define-key ess-mode-map (kbd "M-c M-l")      'ess-load-file); alias, as in 'iES
            (define-key ess-mode-map (kbd "M-c M-v")      'ess-display-help-on-object)
            ;;(define-key ess-mode-map "\C-c5\C-d"'ess-dump-object-into-edit-buffer-other-fr
            (define-key ess-mode-map (kbd "M-c M-s")      'ess-switch-process) ; use a

            ;; different process for the buffer.
```

```
                ;; (define-key map "\C-c\C-t"        'ess-execute-in-tb)
                ;;(define-key ess-mode-map (kbd "M-c \t")     'ess-complete-object-name-deprecat
                ;; (define-key ess-mode-map "\C-c\t"        'comint-dynamic-complete-filename)

                (unless (and (featurep 'emacs) (≥ emacs-major-version 24))
                  (define-key ess-mode-map (kbd "M-c <tab>")  'comint-dynamic-complete))
                (define-key ess-mode-map (kbd "M-c .")        'ess-list-object-completions)

                ;; wrong here (define-key ess-mode-map "\C-c\C-k" 'ess-request-a-process)
                (define-key ess-mode-map (kbd "M-c M-k")      'ess-force-buffer-current)
                (define-key ess-mode-map (kbd "M-c `")        'ess-show-traceback)
                (define-key ess-mode-map (kbd "M-c \\")       'ess-show-call-stack)

                ;;(define-key ess-mode-map (kbd "M-c .")      (lambda () (interactive) (message

                ;;(define-key ess-mode-map "{"                'ess-electric-brace)
                ;;(define-key ess-mode-map "}"                'ess-electric-brace)

                (define-key ess-mode-map (kbd "M-c M-q")      'ess-indent-exp)
                (define-key ess-mode-map (kbd "<M-S-right>")  'ess-mark-function-or-para)
                (if (featurep 'xemacs) ;; work around Xemacs bug (\C-\M-h redefines M-BS):
                    (define-key ess-mode-map [(meta backspace)] 'backward-kill-word))
                ;;(define-key ess-mode-map [delete]           'backward-delete-char-untabify)

                ;;(define-key ess-mode-map "\t"               'ess-indent-or-complete)
                (define-key ess-mode-map (kbd "M-c C-q")      'ess-quit)
                (define-key ess-mode-map (kbd "M-c M-r")      'ess-use-this-dir)

                ;; smart operators; most likely will go in the future into a separate local map
                ;;(define-key map ","          'ess-smart-comma)

                (define-key ess-mode-map (kbd "M-c M-d")       'ess-doc-map)
                (define-key ess-mode-map (kbd "M-c M-e")       'ess-extra-map)
                (define-key ess-mode-map (kbd "M-c M-t")       'ess-dev-map)
                (define-key ess-mode-map (kbd "M-c C-d")       'ess-doc-map)
                (define-key ess-mode-map (kbd "M-c C-e")       'ess-extra-map)
                (define-key ess-mode-map (kbd "M-c C-t")       'ess-dev-map)


;;              (ess-toggle-underscore nil))
                 ;; (define-key ess-mode-map (kbd "M-c M-c")
                 ;;   'ess-eval-region-and-go)
              ))
  (add-hook 'inferior-ess-mode-hook
              #'(lambda nil
```

```
                  (define-key inferior-ess-mode-map [\C-up]
                    'comint-previous-matching-input-from-input)
                  (define-key inferior-ess-mode-map [\C-down]
                    'comint-next-matching-input-from-input)
                  (define-key inferior-ess-mode-map [\C-x \t]
                    'comint-dynamic-complete-filename)
                  )
                )
    :commands R)

  (use-package ess-R-data-view
    :defer t)

  (use-package ess-R-object-popup
    :defer t)

  (use-package ess-R-data-smart-equals
    :disabled t)

  (use-package ess-R-data-smart-underscore
    :disabled t)


#+END_SRC

**** Lang: R/polymode
#+NAME: lang-r.ess.polymode
#+begin_src emacs-lisp

  ;; ---( polymode )-----------------------------------------------------------


(use-package polymode
  :ensure t
  :commands (poly-markdown+r-mode)
  :mode ((("\\.rmd\\'" . poly-markdown+r-mode)
     ("\\.Rmd\\'" . poly-markdown+r-mode))

  :init
  (autoload 'r-mode "ess-site.el" "Major mode for editing R source." t)
)


(use-package poly-markdown
  :ensure t
```

```
  :mode (
     ("\\.md" . poly-markdown-mode)
  )
)

(use-package poly-R
  :ensure t
)


#+END_SRC

*** Lang: Python
**** Lang: Python/mode
#+NAME: lang-python.mode
#+begin_src emacs-lisp

;; ---( python )-----------------------------------------------------------------

;; @see: https://gitlab.com/nathanfurnal/dotemacs/-/snippets/2060535?utm_source=pocket_mylist
;; @see: https://github.com/jidicula/dotfiles/blob/main/init.el?utm_source=pocket_mylist


;; Built-in Python utilities
(use-package python
  :ensure nil
  :hook (python-ts-mode . eglot-ensure)
  :config
  ;; Remove guess indent python message
  (setq python-indent-guess-indent-offset-verbose nil)
  ;; Use IPython when available or fall back to regular Python
  (cond
   ((executable-find "ipython")
    (progn
      (setq python-shell-buffer-name "IPython")
      (setq python-shell-interpreter "ipython")
      (setq python-shell-interpreter-args "-i --simple-prompt")))
   ((executable-find "python3")
    (setq python-shell-interpreter "python3"))
   ((executable-find "python2")
    (setq python-shell-interpreter "python2"))
   (t
    (setq python-shell-interpreter "python")))
  )
```

```
;; Hide the modeline for inferior python processes
(use-package inferior-python-mode
  :ensure nil
  :hook (inferior-python-mode . hide-mode-line-mode))

;; Required to hide the modeline
(use-package hide-mode-line
  :ensure t
  :defer t)



;; (use-package python-mode
;;   :mode ("\\.py\\'" . python-mode)
;;   :interpreter ("python" . python-mode)
;;   :config
;;   (defvar python-mode-initialized nil)
;;   (defun my-python-mode-hook ()
;;     (unless python-mode-initialized
;;       (setq python-mode-initialized t)
;;       (info-lookup-add-help
;;        :mode 'python-mode
;;        :regexp "[a-zA-Z_0-9.]+"
;;        :doc-spec
;;        '(("(python)Python Module Index" )
;;          ("(python)Index"
;;           (lambda
;;             (item)
;;             (cond
;;              ((string-match
;;                "\\([A-Za-z0-9_]+\\)() (in module \\([A-Za-z0-9_.]+\\))" item)
;;               (format "%s.%s" (match-string 2 item)
;;                       (match-string 1 item))))))))
;;     (setq indicate-empty-lines t)
;;     (set (make-local-variable 'parens-require-spaces) nil)
;;     (setq indent-tabs-mode nil)
;;     (bind-key "C-c C-z" 'python-shell python-mode-map)
;;     (unbind-key "C-c c" python-mode-map))
;;   (add-hook 'python-mode-hook 'my-python-mode-hook))



#+END_SRC
```

```
**** Lang: Python/env
#+NAME: lang-python.env
#+begin_src emacs-lisp
;; ---( uv / virtual env integration )-----------------------------------------
;; Architecture:
;; uv-mode - activates the .venv found by uv for shell commands
;; pet - resolves the exact interpreter from pyproject.toml / uv.lock
;; and wires it into LSP before lsp-deferred fires
;; pyvenv-auto - silently activates .venv / venv on buffer/dired entry
;;
;; poetry and pipenv hooks have been removed; poetry tracking caused conflicts
;; with uv-mode and added latency on post-command-hook.

;; ---( uv-mode )---------------------------------------------------------------
;; uv-mode sets PATH / VIRTUAL_ENV for the current project's .venv

(use-package uv-mode
  :ensure t
  :hook ((python-mode . uv-mode-auto-activate-hook)
         (python-ts-mode . uv-mode-auto-activate-hook))
  )


;; ---( pet: Python Executable Tracker )-------------------------------------
;; pet reads uv.lock / pyproject.toml / .python-version to resolve the exact
;; interpreter, then sets buffer-local variables consumed by lsp-pyright.

(use-package pet
  :ensure t
  :config
  (add-hook 'python-base-mode-hook
            (lambda ()
              (setq-local python-shell-interpreter
                          (or (pet-executable-find "python") "python3")
                          python-shell-interpreter-args "-i")
              (pet-mode)))
  ;; Wire pet's resolved venv into lsp-pyright before the server starts.
  (add-hook 'pet-mode-hook
            (lambda ()
              (when-let ((root (pet-virtualenv-root)))
                (setq-local
                 lsp-pyright-venv-path
                 (file-name-directory (directory-file-name root))
                 lsp-pyright-python-executable-cmd
                 (pet-executable-find "python"))))))
```

```
;; ---( pyvenv-auto )-------------------------------------------------------
;; pyvenv-auto activates .venv automatically on project entry; it also
;; triggers pyvenv-post-activate-hooks which restarts the Python process.
(use-package pyvenv-auto
  :ensure t
  :hook ((python-mode . pyvenv-auto-run)
         (python-ts-mode . pyvenv-auto-run)))


;; Keep pyvenv available for manual workon and menu-bar integration.
(use-package pyvenv
  :ensure t
  :defer t
  :config
  ;; Update WORKON_HOME to the uv global venv store.
  (setenv "WORKON_HOME" (expand-file-name "~/.local/share/uv/venvs"))
  (setq pyvenv-menu t))


;; ---( virtual env )-------------------------------------------------------
;; used by dap-mode's dap-python--pyenv-executable-find

(use-package with-venv
  :ensure t)





;; ---( poetry: disabled )-------------------------------------------------
;; poetry tracking hooks have been replaced by pet + uv-mode.
;; The package is kept as disabled so the tangle does not break any
;; residual callers in other files that may still reference it

(use-package poetry
  :ensure t
  :disabled t
  ;; :config
  ;; (add-hook 'poetry-tracking-mode-hook (lambda () (remove-hook 'post-command-hook 'poetry-tra
  ;; (add-hook 'python-mode-hook 'poetry-track-virtualenv)
  ;; (add-hook 'projectile-after-switch-project-hook 'poetry-track-virtualenv)
  )
```

```
#+END_SRC
**** Lang: Python/lsp
#+NAME: lang-python.lsp
#+begin_src emacs-lisp


;; ---( lsp-pyright / basedpyright )-------------------------------------------
;; basedpyright is a community fork of pyright with stricter defaults,
;; inlay hints, and better PEP 695 generics support. Switch the command
;; back to "pyright" if the upstream server is preferred.
;;
;; Install
;; npm install -g basedpyright
;; # or, per-project:
;; uv tool install basedpyright

(use-package lsp-pyright
  :ensure t
  :defer t
  :custom
  (lsp-pyright-langserver-command "basedpyright")
  (lsp-pyright-disable-language-service nil)
  (lsp-pyright-disable-organize-imports nil)
  (lsp-pyright-auto-import-completions t)
  (lsp-pyright-use-library-code-for-types t)
  ;; "workspace" mode scans the full project; use "openFilesOnly" on slow
  ;; machines or when stub sets are very large.
  (lsp-pyright-diagnostic-mode "workspace")
  ;; Type-checking strictness: "standard" for most NLP projects.
  ;; Override per project via pyrightconfig.json or .dir-locals.el.
  (lsp-pyright-type-checking-mode "standard")
  ;; basedpyright inlay hints (no pyrightconfig.json required).
  (lsp-pyright-basedpyright-inlay-hints-variable-types t)
  (lsp-pyright-basedpyright-inlay-hints-function-return-types t)
  (lsp-pyright-basedpyright-inlay-hints-call-argument-names "all")
  :hook
  ((python-mode . (lambda () (require 'lsp-pyright) (lsp-deferred)))
   (python-ts-mode . (lambda () (require 'lsp-pyright) (lsp-deferred)))
   )
  )


;; ---( eglot + pyright + ruff )-----------------------------------------------
(use-package eglot
```

```
  :ensure nil
  :bind (:map eglot-mode-map
              ("C-c c a" . eglot-code-actions)
              ("C-c c r" . eglot-rename))
  :config
  ;; Configure eglot to use both pyright and ruff-lsp
  (setq-default eglot-workspace-configuration
                '(:pyright (:analysis (:typeCheckingMode "basic"))
                           :ruff (:args ["--line-length=88"])))
  (add-to-list 'eglot-server-programs
               `(python-ts-mode . ("pyright-langserver" "--stdio"))))

#+END_SRC

**** Lang: Python/tools
#+NAME: lang-python.tools
#+begin_src emacs-lisp

;; ---( ruff: LSP add-on and formatter )----------------------------------------
;; ruff is integrated at two levels:
;; 1. ruff server - secondary LSP client alongside basedpyright (lint diagnostics)
;; 2. ruff-format - on-save buffer formatter (replaces yapfify + python-black)
;;
;; yapfify and python-black are removed: running three on-save formatters caused
;; race conditions and produced inconsistent results.
;;
;; Install: uv tool install ruff
;; or: pip install ruff (inside the project venv)

;; Register ruff server as an add-on LSP client (diagnostics only, no hover).
(with-eval-after-load 'lsp-mode
  (lsp-register-client
   (make-lsp-client
    :new-connection
    (lsp-stdio-connection
     (lambda () (list (or (executable-find "ruff") "ruff") "server")))
    :activation-fn (lsp-activate-on "python")
    :server-id 'ruff-lsp
    ;; add-on? t: run alongside basedpyright, not instead of it.
    :add-on? t
    :initialization-options
    '(:settings (:logLevel "warning"
                           :lint (:enable t)
                           :format (:enable t))))))
```

```
;; ruff-format: on-save formatting.
;; Replaces python-black-on-save-mode and yapf-mode.

(use-package ruff-format
  :ensure t
  :hook ((python-mode . ruff-format-on-save-mode)
         (python-ts-mode . ruff-format-on-save-mode)))

;; py-isort: sort imports before save.
;; Retained because ruff's isort implementation does not yet cover all
;; isort profiles (e.g. google, wemake). Remove if ruff's [tool.ruff.lint.isort]
;; is sufficient for the project.

;; (use-package py-isort
;;   :ensure t
;;   :after python
;;   :hook (before-save . py-isort-before-save))

  ;; ---( yapfify )-------------------------------------------------------------
  ;; yapfify: superseded by ruff-format.

  ;; (use-package yapfify
  ;;   :ensure nil
  ;;   :disabled t
  ;;   :hook (python-mode . yapf-mode))


  ;; ---( python-black )-------------------------------------------------------------------
  ;; python-black: superseded by ruff-format.

  ;; (use-package python-black
  ;;   ;;:delight python-black-on-save-mode "⍇"
  ;;   :ensure nil
  ;;   :hook
  ;;   (python-mode . python-black-on-save-mode)
  ;;   :init
  ;;   (put 'python-black-command 'safe-local-variable #'stringp)
  ;;   (put 'python-black-extra-args 'safe-local-variable #'stringp)
  ;;   (put 'python-black-on-save-mode 'safe-local-variable #'booleanp)
  ;;   )

#+END_SRC

**** Lang: Python/test
```

```
#+NAME: lang-python.test
#+begin_src emacs-lisp

;; ---( pytest )-----------------------------------------------------------------
;; python-pytest uses transient (same UX as magit) and detects treesit
;; automatically for function/class DWIM commands.
;;
;; Note: "C-c t" was previously bound to treemacs (line 2717).
;; The binding below uses "C-c T" (uppercase) to avoid that conflict.
;; Adjust according to preference; see treemacs binding at line 2717.

(use-package python-pytest
  :ensure t
  :after python
  :commands (python-pytest-dispatch
             python-pytest
             python-pytest-file
             python-pytest-file-dwim
             python-pytest-function
             python-pytest-function-dwim
             python-pytest-last-failed
             python-pytest-repeat)
  :custom
  (python-pytest-confirm nil)
  ;; Default flags: colour output, most-recently-failed first, compact traceback.
  (python-pytest-arguments '("--color=yes" "--failed-first" "--tb=short"))
  :config
  ;; Integration with uv: ensure pytest is run via 'uv run'
  (setq python-pytest-executable "uv run pytest")
  :bind (:map python-base-mode-map
              ("C-c t d" . python-pytest-dispatch)
              ("C-c t t" . python-pytest)
              ("C-c t f" . python-pytest-file-dwim)
              ("C-c t F" . python-pytest-file)
              ("C-c t m" . python-pytest-function-dwim)
              ("C-c t M" . python-pytest-function)
              ("C-c t l" . python-pytest-last-failed)
              ("C-c t r" . python-pytest-repeat))
 )


;; ---( coverage overlay )-------------------------------------------------------
;; cov reads coverage.json / .coverage and paints hit/miss fringe indicators.
;; Coverage is optional; generate data with:
;; uv run pytest --cov --cov-report=json
```

```
;; then M-x cov-mode in any Python buffer.

(use-package cov
  :ensure t
  :defer t
  :custom
  (cov-coverage-file-paths '("." "coverage" "htmlcov"))
  ;; 'coverage-py reads .coverage directly; switch to 'lcov for lcov output.
  (cov-coverage-mode 'coverage-py))



#+END_SRC

**** Lang: Python/bdd
#+NAME: lang-python.bdd
#+begin_src emacs-lisp

;; ---( behave BDD runner )-------------------------------------------------------
;; No dedicated MELPA package exists for behave as of 2026.
;; This block provides a compilation-mode wrapper with:
;; - a behave-specific error regexp for .feature:LINE navigation
;; - three interactive commands: suite, file-dwim, scenario-at-point
;; - C-c B prefix keybindings (uppercase B avoids the citar C-c b bindings
;; defined at line 7545
;;
;; gherkin-mode provides .feature syntax highlighting.
;; Install: uv add behave --dev (or uv tool install behave

(defgroup python-behave nil
  "Run behave BDD tests from Emacs."
  :group 'python
  :prefix "python-behave-")

(defcustom python-behave-command "behave"
  "Command used to invoke behave."
  :type 'string
  :group 'python-behave)

(defcustom python-behave-arguments '("--no-capture" "--color")
  "Default arguments passed to behave."
  :type '(repeat string)
  :group 'python-behave)

(defcustom python-behave-features-directory "features"
```

```
  "Relative path to the features directory from the project root."
  :type 'string
  :group 'python-behave)

;; Register behave's scenario-location lines in compilation output.
;; Format: features/my_feature.feature:12
(with-eval-after-load 'compile
  (add-to-list 'compilation-error-regexp-alist-alist
               '(behave
                 "\\(features/[^ \t\n]+\\.feature\\):\\([0-9]+\\)"
                 1 2 nil 0 1))
  (add-to-list 'compilation-error-regexp-alist 'behave))

(defun python-behave--project-root ()
  "Return the project root, preferring projectile then project.el."
  (or (and (fboundp 'projectile-project-root)
           (ignore-errors (projectile-project-root)))
      (when-let ((p (project-current)))
        (project-root p))
      default-directory))

(defun python-behave--build-command (&optional target)
  "Construct the behave invocation string.
When TARGET is non-nil it is appended as the positional argument."
  (string-join
   (flatten-list
    (list python-behave-command
          python-behave-arguments
          (when target (list target))))
   " "))

;;;^^###autoload
(defun python-behave ()
  "Run the full behave test suite from the project root."
  (interactive)
  (let ((default-directory (python-behave--project-root)))
    (compile (python-behave--build-command))))


;;;^^###autoload
(defun python-behave-feature-dwim ()
  "Run behave on the feature file associated with the current buffer.
Falls back to the full suite when no feature file can be inferred."
  (interactive)
  (let* ((default-directory (python-behave--project-root))
```

```
         (target
          (cond
           ;; Current buffer is a .feature file.
           ((and buffer-file-name
                 (string-suffix-p ".feature" buffer-file-name))
            (file-relative-name buffer-file-name default-directory))
           ;; Current buffer is a steps file: run the whole feature dir.
           ((and buffer-file-name
                 (string-match-p "/steps/" buffer-file-name))
            python-behave-features-directory)
           (t nil))))
    (compile (python-behave--build-command target))))

;;;^^###autoload
(defun python-behave-scenario-at-point ()
  "Run behave for the scenario whose name is at or before point.
Uses behave's --name flag to select the scenario."
  (interactive)
  (let* ((default-directory (python-behave--project-root))
         (scenario-name
          (save-excursion
            (when (re-search-backward
                   "^\\s-*Scenario\\(^?:\\s-+Outline\\)^?:\\s-+\\(.*\\)$"
                   nil t)
              (match-string-no-properties 1))))
         (cmd (if scenario-name
                  (concat (python-behave--build-command)
                          " --name "
                          (shell-quote-argument scenario-name))
                (python-behave--build-command))))
    (compile cmd)))

;; Keybindings: C-c B prefix (uppercase avoids conflict with citar C-c b).
(with-eval-after-load 'python
  (define-key python-base-mode-map (kbd "C-c B b") #'python-behave)
  (define-key python-base-mode-map (kbd "C-c B f") #'python-behave-feature-dwim)
  (define-key python-base-mode-map (kbd "C-c B s") #'python-behave-scenario-at-point))
;; gherkin-mode: .feature syntax highlighting.
;; Falls back to conf-mode if the package is unavailable.
(use-package gherkin-mode
  :ensure t
  :mode "\\.feature\\'")
```

```
#+END_SRC

**** Lang: Python/elpy
#+NAME: lang-python.elpy
#+begin_src emacs-lisp


  ;; ---( python: elpy )-------------------------------------------------------------

  (use-package elpy
    :disabled t
    :preface

    ;; @see: https://elpy.readthedocs.org/en/latest/
    ;; @see: https://github.com/jorgenschaefer/elpy
    ;; @see: https://youtu.be/0kuCeS-mfyc

    (defvar elpy-mode-map
      (let ((map (make-sparse-keymap)))
        ;; Alphabetical order to make it easier to find free C-c C-X
        ;; bindings in the future. Heh.

        ;; (define-key map (kbd "<backspace>") 'python-indent-dedent-line-backspace)
        ;; (define-key map (kbd "<backtab>")   'python-indent-dedent-line)

        ;; (define-key map (kbd "C-M-x")   'python-shell-send-defun)

        (define-key map (kbd "M-c <")   'python-indent-shift-left)
        (define-key map (kbd "M-c >")   'python-indent-shift-right)

        (define-key map (kbd "M-c RET") 'elpy-importmagic-add-import)
        (define-key map (kbd "M-c M-b") 'elpy-nav-expand-to-indentation)
        (define-key map (kbd "M-c M-c") 'elpy-shell-send-region-or-buffer)
        (define-key map (kbd "M-c M-d") 'elpy-doc)
        (define-key map (kbd "M-c M-e") 'elpy-multiedit-python-symbol-at-point)
        (define-key map (kbd "M-c M-f") 'elpy-find-file)
        (define-key map (kbd "M-c M-n") 'elpy-flymake-next-error)
        (define-key map (kbd "M-c M-o") 'elpy-occur-definitions)
        (define-key map (kbd "M-c M-p") 'elpy-flymake-previous-error)
        (define-key map (kbd "M-c M-s") 'elpy-rgrep-symbol)
        (define-key map (kbd "M-c M-t") 'elpy-test)
        (define-key map (kbd "M-c M-v") 'elpy-check)
        (define-key map (kbd "M-c M-z") 'elpy-shell-switch-to-shell)
```

```
    (define-key map (kbd "M-c M-r i") 'elpy-importmagic-fixup)
    (define-key map (kbd "M-c M-r p") 'elpy-autopep8-fix-code)
    (define-key map (kbd "M-c M-r r") 'elpy-refactor)

    ;; (define-key map (kbd "<S-return>") 'elpy-open-and-indent-line-below)
    ;; (define-key map (kbd "<C-S-return>") 'elpy-open-and-indent-line-above)

    ;; (define-key map (kbd "<C-return>") 'elpy-shell-send-current-statement)

    ;; (define-key map (kbd "<C-down>") 'elpy-nav-forward-block)
    ;; (define-key map (kbd "<C-up>") 'elpy-nav-backward-block)
    ;; (define-key map (kbd "<C-left>") 'elpy-nav-backward-indent)
    ;; (define-key map (kbd "<C-right>") 'elpy-nav-forward-indent)

    ;; (define-key map (kbd "<M-down>") 'elpy-nav-move-line-or-region-down)
    ;; (define-key map (kbd "<M-up>") 'elpy-nav-move-line-or-region-up)
    ;; (define-key map (kbd "<M-left>") 'elpy-nav-indent-shift-left)
    ;; (define-key map (kbd "<M-right>") 'elpy-nav-indent-shift-right)

    ;; (define-key map (kbd "M-.")     'elpy-goto-definition)
    ;; (define-key map (kbd "M-TAB")   'elpy-company-backend)

    (define-key map (kbd "<C-S-return>") 'elpy-open-and-indent-line-below)
    ;;(define-key map (kbd "<C-S-return>") 'elpy-open-and-indent-line-above)

    ;;(define-key map (kbd "<C-return>") 'elpy-shell-send-current-statement)

    (define-key map (kbd "<M-right>") 'elpy-nav-forward-block)
    (define-key map (kbd "<M-left>") 'elpy-nav-backward-block)
    ;; (define-key map (kbd "<C-S-left>") 'elpy-nav-backward-indent)
    ;; (define-key map (kbd "<C-S-right>") 'elpy-nav-forward-indent)

    ;; (define-key map (kbd "<M-S-down>") 'elpy-nav-move-line-or-region-down)
    ;; (define-key map (kbd "<M-S-up>") 'elpy-nav-move-line-or-region-up)
    (define-key map (kbd "<M-S-left>") 'elpy-nav-indent-shift-left)
    (define-key map (kbd "<M-S-right>") 'elpy-nav-indent-shift-right)

    (define-key map [(meta prior)]    'elpy-goto-definition)
    (define-key map [(meta next)]     'pop-tag-mark)

    (define-key map [(control menu)]   'elpy-company-backend)

    map)
  "Key map for the Emacs Lisp Python Environment.")
:config
```

```
    (elpy-enable)
    (setq python-shell-interpreter "jupyter"
          python-shell-interpreter-args "console --simple-prompt")

    ;; (elpy-use-ipython "ipython3")
    (defalias 'workon 'pyvenv-workon))

  (setenv "PYTHONIOENCODING" "utf-8")
  (add-to-list 'process-coding-system-alist '("python" . (utf-8 . utf-8)))
  (add-to-list 'process-coding-system-alist '("elpy" . (utf-8 . utf-8)))
  (add-to-list 'process-coding-system-alist '("flake8" . (utf-8 . utf-8)))

#+END_SRC

**** Lang: Python/ein
#+NAME: lang-python.ein
#+begin_src emacs-lisp

  ;; ---( python: ein )-------------------------------------------------------------


  (use-package ein
    :unless (version< emacs-version "25.1")
    ;; :defer t
    ;;:ensure t
    :disabled t
    :init
    (progn
      (with-eval-after-load 'ein-notebooklist
        (define-key ein:notebooklist-mode-map (kbd "<S-return>") 'ein:worksheet-execute-cell-and
        (define-key ein:notebooklist-mode-map (kbd "<C-return>") 'ein:worksheet-execute-cell)
        ))
    :config
    (defalias 'eip 'ein:notebooklist-open))



  ;; (use-package ein
  ;;   :unless (version< emacs-version "25.1")
  ;;   :ensure t
  ;;   :defer t
  ;;   :commands ein:notebooklist-open
  ;;   :init
  ;;   ;; (progn
  ;;   ;;   (with-eval-after-load 'ein-notebooklist
```

```
;;   ;;     ;; removing keybindings
;;   ;;     (define-key ein:notebook-mode-map (kbd "M-p") nil)
;;   ;;     (define-key ein:notebook-mode-map (kbd "<M-up>") nil)
;;   ;;     (define-key ein:notebook-mode-map (kbd "<M-down>") nil)
;;   ;;     ;; changing keybinding
;;   ;;     (define-key ein:notebook-mode-map (kbd "C-s") 'ein:notebook-save-notebook-command)
;;   ;;     (define-key ein:notebook-mode-map (kbd "<M-S-up>") 'ein:worksheet-move-cell-up)
;;   ;;     (define-key ein:notebook-mode-map (kbd "<M-S-down>") 'ein:worksheet-move-cell-down
;;   :config
;;   (defalias 'einp 'ein:notebooklist-open)
;;   (defalias 'eins 'ein:jupyter-server-start)
;;   )

;; ---( python: 0mq )-------------------------------------------------------------

;; @see: https://github.com/nnicandro/emacs-zmq
;; @see: https://github.com/nnicandro/emacs-zmq/issues/48
;; dnf install zeromq-devel
;; apt install libczmq-dev


;; python and jupyter
;;; custom zmq build - see https://github.com/alexmurray/emacs-snap/issues/66
;;; @see: https://github.com/martibosch/snakemacs/blob/main/main.el#L346

(cond ((getenv "EMACS_SNAP_DIR")

       (let* ((emacs-snap-dir (file-name-as-directory (getenv "EMACS_SNAP_DIR")))
              (process-environment (append process-environment `(,(concat "CC=" emacs-snap-dir
                                                                  ,(concat "CXX=" emacs-snap-di
                                                                  ,(concat "CFLAGS=--sysroot="
                                        ,(concat "CPPFLAGS=--sysroot=" emacs-snap-dir)
                                        ,(concat "LDFLAGS=--sysroot=" emacs-snap-dir " -L" emacs-
         (use-package zmq
          :if (h7/use-py-jupyter)
          :defer t
          :ensure t)
         ))
      (t
        (use-package zmq
          :if (h7/use-py-jupyter)
          :defer t
          :ensure t)
        ))
```

```
  ;; (use-package zmq
  ;;   :if (h7/use-py-jupyter)
  ;;   :defer t
  ;;   :ensure t
  ;;   :preface

;;     (package-install "zmq")
;;     (vterm)

;; cd ~/.emacs.d/elpa

;; ls -lda zmq*
;; cd      zmq*


;; ES=${EMACS_SNAP_DIR:-/snap/emacs/current}
;; export CC=${ES}/usr/bin/gcc-10
;; export CXX=${ES}/usr/bin/g++-10
;; export CFLAGS=--sysroot=${ES}
;; export LDFLAGS="--sysroot=${ES} -L${ES}/usr/lib"

;; printenv | grep -i -e ^cc= -e ^cxx= -e ^cflags= -e ^ldflags=
;; ls -l $CC $CXX

;; make all

;;     (package-install "jupyter")

    ;; :init
    ;; :config
    ;; )

  ;; (use-package zmq
  ;;   :ensure t
  ;;   :preface
  ;;   (when (getenv "EMACS_SNAP_DIR")
  ;;     (unless (directory-files-recursively (concat user-emacs-directory "") "zmq-.*\\.so$" ni
  ;;              (progn
  ;;                ;; @see: https://github.com/nnicandro/emacs-zmq/issues/48
  ;;                (let* ((emacs-snap-dir (file-name-as-directory (getenv "EMACS_SNAP_DIR")))
  ;;                       (process-environment
  ;;                        (append `(,(concat "CC=" emacs-snap-dir "usr/bin/gcc-10" )
  ;;                                  ,(concat "CXX=" emacs-snap-dir "usr/bin/g++-10")
```

```
;;                                  ,(concat "CFLAGS=--sysroot=" emacs-snap-dir " -B" emacs-sn
;;                                  ,(concat "CPATH=" (file-name-directory (car (file-expand-w
;;                    ,(concat "CPPFLAGS=--sysroot=" emacs-snap-dir)
;;                    ,(concat "LDFLAGS=--sysroot=" emacs-snap-dir " -L" emacs-snap-dir "usr/li
;;                                  ,(concat "PKG_CONFIG_PATH=" (car (file-expand-wildcards (c
;;                                process-environment)))
;;                   ;; @see: https://github.com/nnicandro/emacs-zmq/issues/48#issuecomment-220
;;                   (when (fboundp 'native-compile-async)
;;                     (progn
;;                       (setq native-comp-deferred-compilation t
;;                             native-comp-deferred-compilation-deny-list
;;                             '("/mu4e.*\\.el$" "jupyter" "zmq" "eaf" "eaf-mode" "emacs-zmq"))
;;                ;; (custom-set-variables
;;                ;;  '(native-comp-async-report-warnings-errors 'silent))
;;                ;; ;; (let ((snap (file-name-as-directory "/snap/emacs/current")))
;;                ;; ;;    (setq-default native-comp-driver-options (list (concat "--sysroot="
;;                ;; ;;                                                   (concat "-B" snap "usr

;;                   (load-library "zmq")

;;                   ))))
;;   :init
;;   :config
;;   )


;; ---( python: jupyter )-------------------------------------------------------------

;; @see: https://sqrtminusone.xyz/posts/2021-05-01-org-python/

(use-package jupyter
  :if (h7/use-py-jupyter)
  :defer t
  :ensure t
  :init
  :config
  )


;; ---( python: code cells )----------------------------------------------------------------

;; @see: https://github.com/martibosch/snakemacs/blob/main/main.el#L444

(use-package code-cells
  :ensure t
```

```
    :after org
    :config
    (setq code-cells-convert-ipynb-style '(("pandoc" "--to" "ipynb" "--from" "org")
                        ("pandoc" "--to" "org" "--from" "ipynb")
                        org-mode))
    ;; see https://github.com/astoff/code-cells.el/issues/22
    ;; (defun gm/jupyter-eval-region (beg end)
    ;;   (jupyter-eval-region nil beg end))
    ;; (add-to-list 'code-cells-eval-region-commands '(jupyter-repl-interaction-mode . gm/jupyte
    (let ((map code-cells-mode-map))
      (define-key map (kbd "C-c <up>") 'code-cells-backward-cell)
      (define-key map (kbd "C-c <down>") 'code-cells-forward-cell)
      (define-key map (kbd "M-<up>") 'code-cells-move-cell-up)
      (define-key map (kbd "M-<down>") 'code-cells-move-cell-down)
      (define-key map (kbd "C-c C-c") 'code-cells-eval)
      ;; Overriding other minor mode bindings requires some insistence...
      (define-key map [remap jupyter-eval-line-or-region] 'code-cells-eval)))
      (defun my/new-notebook (notebook-name &optional kernel)
        "Creates an empty notebook in the current directory with an associated kernel."
        (interactive "sEnter the notebook name: ")
        (when (file-name-extension notebook-name)
          (setq notebook-name (file-name-sans-extension notebook-name)))
        (unless kernel
          (setq kernel (jupyter-kernelspec-name (jupyter-completing-read-kernelspec))))
        (unless (executable-find "jupytext")
          (error "Can't find \"jupytext\""))
        (let ((notebook-py (concat notebook-name ".py")))
          (shell-command (concat "touch " notebook-py))
          (shell-command (concat "jupytext --set-kernel " kernel " " notebook-py))
          (shell-command (concat "jupytext --to notebook " notebook-py))
          (shell-command (concat "rm " notebook-py))
          (message (concat "Notebook successfully created at " notebook-name ".ipynb"))))



#+END_SRC

*** Lang: Julia
#+NAME: lang-julia
#+begin_src emacs-lisp

;; ---( julia )-----------------------------------------------------------------

;; @see: https://github.com/JuliaEditorSupport/julia-emacs
;; @see: https://github.com/tpapp/julia-repl
```

```
;; @see: https://github.com/nnicandro/emacs-jupyter
;; @see: https://julia-users-zurich.github.io/talks/talk-2018-04/emacs.html
;; @see: https://github.com/cgroll/dot_emacs.d/blob/master/init.el

(use-package julia-mode
  :ensure t
  :defer t
  :commands julia-mode
  :mode ("\\.jl$" . julia-mode)
  :init
  (progn
    (autoload 'julia-mode "julia-mode" nil t)
    (setq inferior-julia-program-name "julia")
    )
  :config
  (progn
    (setq inferior-julia-program-name "julia")
    )
  )


(use-package julia-repl
  :ensure t
  :defer t
  :config
  (progn
    (add-to-list 'julia-mode-hook 'julia-repl-mode)
    )
  )

(use-package lsp-julia
  :ensure t
  :hook ((
          julia-mode
          ) . lsp-mode)
  ;;:config
  ;;(setq lsp-julia-default-environment "~/.julia/environments/v1.7")
  )

;; ;; allow julia to be loaded through call to julia-mode or
;; ;; ess-inferior process
;; ;; follow-ups: etags?
;; (use-package julia-mode
;;    :defer t
;;    :commands julia-mode
```

```
;;    :mode ("\\.jl$" . julia-mode)
;;    :init
;;    (progn
;;       (autoload 'julia-mode "julia-mode" nil t)
;;       (setq inferior-julia-program-name "/usr/bin/julia")
;;       )
;;    :config
;;    (progn
;;       (add-to-list 'julia-mode-hook 'cg/modify-current-syntax-table)
;;       (setq inferior-julia-program-name "/usr/bin/julia")
;;       (add-to-list 'julia-mode-hook 'cg/command-line-keybindings)
;;       ;; (add-to-list 'inferior-ess-mode-hook 'cg/command-line-keybindings)
;;       )
;;    )

;; (use-package ess-julia.el
;;    :defer t
;;    :commands julia
;;    :init                                ; run before actual loading
;;    (progn
;;       (autoload 'julia "ess-julia.el" nil t)
;;       (setq inferior-julia-program-name "/usr/bin/julia")
;;       )
;;    :config
;;    (progn
;;       (require 'ess-site)
;;       (setq inferior-julia-program-name "/usr/bin/julia")
;;       (setq ess-tracebug-prefix "\M-c")   ; define debug-mode starting key
;;       (setq ess-use-tracebug t)           ; tracebug is called for R
;;                                           ; AND JULIA!!
;;       (setq ess-tracebug-inject-source-p t)
;;       (add-to-list 'julia-mode-hook 'cg/command-line-keybindings)
;;       ;; (add-to-list 'inferior-ess-mode-hook 'cg/command-line-keybindings)
;;       )
;;    )
;; ;; in order to add ess-process afterward, apply julia-mode again on
;; ;; open buffers - probably ess-julia.el has to be loaded again also:
;; ;; M-x load-file ess-julia.el

#+END_SRC


*** Lang/end
#+NAME: lang-end
#+begin_src emacs-lisp
```

```
  ;; }}}  .lang

#+END_SRC


** Keys
*** keys/develop

#+begin_src emacs-lisp

;; ============================================
;; ---( Function Keys )-----
;; ============================================
(message "SITE:K-FUNKEYS")


;; ---( F1: Help )-------------------------------------------------------


(global-set-key [f1] 'help )
;; (global-set-key [(shift f1)] 'woman )
(global-set-key [(control f1)] 'find-function )
(global-set-key [(meta f1)] 'function-key-error)
(global-set-key [(shift meta f1)] 'function-key-error)


;; ---( F2: Bookmarks/Breakpoints )-------------------------------------

(global-set-key [f2] 'dashboard-open)
(global-set-key [(shift f2)] 'bookmark-set )

;; (global-set-key [(control f2)]
;;     #'(lambda () (interactive)
;;        (if (eq hs-minor-mode nil)
;;     (progn
;;       (hs-minor-mode t)
;;       (hs-hide-all))
;;          (progn
;;            (hs-toggle-hiding)
;;     (hs-minor-mode nil)))))

;; (global-set-key [(control f2)]
;;     #'(lambda () (interactive)
;;          (progn
```

```
;;             ;;(vimish-fold-delete-all)
;;             (vimish-fold-from-marks)
;;      (vimish-fold-toggle))))
;; ;; (global-set-key [(shift control f2)]
;; ;;      #'(lambda () (interactive)
;; ;;           (progn
;; ;;             (vimish-fold-from-marks)
;; ;;             ;; (outline-hide-sublevels 1)
;; ;;       (vimish-fold-refold-all))))

(global-set-key [(control f2)]
    #'(lambda () (interactive)
         (progn
           (outline-hide-sublevels 1))))

(global-set-key [(shift control f2)]
    #'(lambda () (interactive)
         (progn
           (outline-show-all))))

(global-set-key [(meta f2)] 'bookmark-bmenu-list)
(global-set-key [(hyper f2)]    'bookmark-bmenu-list)
(global-set-key [(shift meta f2)]
    #'(lambda () (interactive)
        (progn
          (toggle-line-wrapping)
      (linum-mode 'toggle))))


;; ---( F3: ISearch/Find )-----------------------------------------------------

(global-set-key [f3] 'isearch-repeat-forward )
(global-set-key [(shift f3)] 'isearch-repeat-backward )
(global-set-key [(control f3)] 'isearch-iforward )
(global-set-key [(meta f3)] 'occur )
(global-set-key [(shift meta f3)] 'function-key-error)

;; ---( F4: Fold )-----------------------------------------------------

(global-set-key [f4] 'vimish-fold-toggle) ;; or 'vimish-fold is on selected regzion
(global-set-key [(shift f4)] 'hydra-fold/body)
(global-set-key [(control f4)] 'vimish-fold-refold-all-from-marks)
(global-set-key [(shift control f4)] 'vimish-fold-unfold-all)
(global-set-key [(shift meta f4)] 'vimish-fold)
;; (global-set-key [f4] 'call-last-kbd-macro)
```

```
;; (global-set-key [(shift f4)] 'start-or-end-kbd-macro )
;; (global-set-key [(control f4)] 'edit-last-kbd-macro )
;; (global-set-key [(meta f4)] 'kbd-macro-query )
;; (global-set-key [(shift meta f4)] 'edit-last-kbd-macro )
;; ;; (global-set-key [(control f4)] 'start-kbd-macro )
;; ;; (global-set-key [(meta f4)] 'end-kbd-macro )

;; ---( F5: Search/Grep )-----------------------------------------------------

(global-set-key [f5] 'isearch-forward-regexp )
(global-set-key [(shift f5)] 'isearch-backward-regexp )
(global-set-key [(control f5)] 'find-grep-dired )
(global-set-key [(meta f5)] 'grep  )
(global-set-key [(shift meta f5)] 'function-key-error)

;; ---( F6: Replace/Ediff )-------------------------------------------------------

(global-set-key [f6] 'query-replace )
(global-set-key [(shift f6)] 'query-replace-regexp )
(global-set-key [(control f6)] 'compare-windows )
(global-set-key [(meta f6)] 'ediff )
(global-set-key [(shift meta f6)] 'function-key-error)

;; ---( F7: Debug/Step )----------------------------------------------------

;; (global-set-key [f7] 'gud-step ) ;;@TODO: move to local mode map
;; ;; (global-set-key [(control f7)] 'function-key-error)
;; (global-set-key [(meta f7)] 'function-key-error)
;; (global-set-key [(shift f7)] 'function-key-error)
;; (global-set-key [(shift meta f7)] 'function-key-error)

;; ---( F8: Debug/Next )----------------------------------------------------

;; ;;(global-set-key [f8] 'function-key-error ) ;;WM expose
;; (global-set-key [(control f8)] 'gud-next) ;;@TODO: move to local mode map
;; (global-set-key [(meta f8)] 'function-key-error)
;; (global-set-key [(shift f8)] 'function-key-error)
;; (global-set-key [(shift meta f8)] 'function-key-error)

;; ---( F9: compile/run )---------------------------------------------------

;;(global-set-key [f9] 'function-key-error ) ;;WM expose
;;(global-set-key [f9] 'perldb ) ;;@TODO: move to local mode map
(global-set-key [(f9)] 'eshell-toggle )
(global-set-key [(shift f9)] 'eshell-here )
```

```
(cond
 ((fboundp 'vterm);;
  (progn
    (message "vterm:bind [C-u] [C-u] C-F9")
    (global-set-key [(control f9)] 'vterm-here )
    ))
 ((fboundp 'multi-term);;
  (progn
    (global-set-key [(control f9)] 'multi-term )
    ))
 (t ;; fallback to VC bindings
  (progn
    (global-set-key [(control f9)] 'ansi-term )
    ))
 )

(global-set-key [(shift meta f9)] 'projectile-run-vterm )

;;(global-set-key [(shift meta f9)] 'mode-compile-kill )
;;(global-set-key [(meta f9)] 'mode-compile )
;;(global-set-key [(meta f9)] 'recompile )
;;(global-set-key [(shift meta f9)] 'compile)
;;(global-set-key [(meta f9)] 'shell)
(global-set-key [(meta f9)] 'list-processes)


;; ---( F10: UI )-----------------------------------------------------

;;(global-set-key [f10] 'menu )
;;(global-set-key [(control f10)] 'menu-bar-mode )
(global-set-key [(control f10)] 'toggle-menubar )
(global-set-key [(shift f10)] 'toggle-toolbar )
(global-set-key [(meta f10)] 'speedbar )
(global-set-key [(hyper f10)] 'treemacs )
(global-set-key [(shift meta f10)] 'toggle-gutter)


;; ---( F11: VCS )----------------------------------------------------

(cond
 ((fboundp 'magit-status);; Git magit
  (progn
    (global-set-key [f11] 'magit-status )
    (global-set-key [(shift meta f11)] 'vc-next-action)
```

```
    (global-set-key [(meta f11)] 'vc-diff )
    (global-set-key [(control meta f11)] 'toggle-fullscreen )
    ))
 (t ;; fallback to VC bindings
  (progn
    (global-set-key [f11] 'vc-next-action )
    (global-set-key [(shift f11)] 'vc-annotate )
    (global-set-key [(control f11)] 'vc-directory )
    (global-set-key [(meta f11)] 'vc-version-diff )
    (global-set-key [(shift meta f11)] 'function-key-error)
    (global-set-key [(control meta f11)] 'toggle-fullscreen )
    ))
 )




;;(define-key speedbar-key-map [button1] 'dframe-click)


;; ---( F12: Frames )-----------------------------------------------------

;; (global-set-key [f12] 'make-frame )
(global-set-key [f12] 'h7/make-note-frame )
(global-set-key [(control f12)] 'delete-frame )
(global-set-key [(shift f12)] 'buffer-menu )
(global-set-key [(meta f12)] 'toggle-frame-maximized )
(global-set-key [(shift meta f12)] 'function-key-error)


#+END_SRC
```